



UNIVERSIDAD DE MÁLAGA



E.T.S. INGENIERÍA
INFORMÁTICA
UNIVERSIDAD DE MÁLAGA

Doble Grado Matemáticas e Ingeniería Informática

Mención en Computación

Análisis y predicción de velocidades y tiempos de Fórmula 1 mediante inteligencia artificial

Analysis and prediction of speeds and lap times in Formula 1 using artificial intelligence

Realizado por

Juan Elías Lara Soto

Tutorizado por

Dr. Miguel Ángel Molina Cabello

Co-tutorizado por

Dr. Ezequiel López Rubio

Departamento

Departamento de Lenguajes y Ciencias de la Computación

Málaga, Febrero, 2026

Resumen

El siguiente trabajo de fin de grado se centra en el análisis y modelado de datos reales de Fórmula 1 con el objetivo de predecir velocidades y tiempos de vuelta. Para ello, se emplea la telemetría de los vehículos recogida en la librería FastF1, que permite acceder a información detallada de las sesiones oficiales de competición. A partir de estos datos se realiza un proceso de extracción, limpieza y análisis exploratorio para estudiar el comportamiento de los monoplazas y la variabilidad de la velocidad a lo largo del circuito. Posteriormente, se aplican técnicas de modelado basadas en series temporales¹, como ARIMA, con el fin de estimar valores futuros de velocidad o rendimiento a partir de instantes temporales previos. Finalmente, se evalúa el modelo desarrollado y se analizan los resultados obtenidos, extrayendo conclusiones sobre la relación entre el trazado del circuito, la dificultad de sus curvas y el rendimiento del monoplaza. De este modo, el trabajo contribuye al análisis académico del rendimiento en Fórmula 1 mediante un enfoque reproducible basado en datos reales de competición.

Palabras clave: Fórmula 1, telemetría, análisis de datos, series temporales, predicción, ARIMA

¹Una serie temporal es una secuencia de observaciones de una variable registradas de forma ordenada en el tiempo.

Abstract

This Bachelor's Thesis focuses on the analysis and modeling of real Formula 1 data with the aim of predicting speeds and lap times. To this end, vehicle telemetry data obtained through the FastF1 library are used, which provide access to detailed information from official competition sessions. Based on these data, a process of extraction, cleaning, and exploratory analysis is carried out in order to study the behavior of the cars and the variability of speed throughout the circuit. Subsequently, time series²-based modeling techniques, such as ARIMA, are applied to estimate future values of speed or performance from previous temporal instants. Finally, the developed model is evaluated and the obtained results are analyzed, drawing conclusions about the relationship between the circuit layout, the difficulty of its corners, and the performance of the car. In this way, the thesis contributes to the academic analysis of Formula 1 performance through a reproducible approach based on real competition data.

Keywords: Formula 1, telemetry, data analysis, time series, prediction, ARIMA

²A time series is a sequence of observations of a variable recorded in chronological order over time.

Agradecimientos

Agradezco ... (por completar)

Índice General

Resumen	1
Abstract	3
Agradecimientos	5
Índice General	7
Índice de Figuras	11
Índice de Tablas	13
1 Introducción	15
1.1 Motivación	16
1.2 Objetivos	16
1.2.1 Objetivo general	16
1.2.2 Objetivos específicos	17
1.3 Tecnologías y herramientas	17
1.4 Estructura de la memoria	20
2 Estado del Arte	23
2.1 Soluciones profesionales	23
2.1.1 Racewatch	24
2.1.2 F1 Insights	25
2.2 Soluciones académicas	26
2.3 Fundamentos metodológicos	27
3 Metodología de trabajo	29
3.1 Diseño y organización del código	30
3.2 Fuente de los datos	31
3.3 Planificación	32
4 Implementación	33
4.1 Extracción y limpieza de datos	33
4.1.1 Introducción y objetivos	33
4.1.2 Información proporcionada por FastF1	34
4.1.3 Arquitectura del pipeline	35

4.1.4	Carga de la sesión: <code>sesion.py</code>	37
4.1.5	Limpieza de tiempos de vuelta: <code>vueltas.py</code>	37
4.1.6	Extracción y limpieza de telemetría: <code>telemetria.py</code>	38
4.1.7	Condiciones meteorológicas: <code>condiciones_meteorologicas.py</code>	42
4.1.8	Geometría y longitud del circuito: <code>circuito.py</code>	42
4.1.9	Persistencia de resultados: <code>almacenar_csv.py</code>	42
4.1.10	Artefactos generados	43
4.1.11	Limitaciones	44
4.2	Análisis Exploratorio de Datos	44
4.2.1	Rendimiento por vuelta (<code>estadisticas_vueltas.py</code>)	45
4.2.2	Telemetría: velocidad, acelerador, freno y marchas (<code>estadisticas_teleme- tria.py</code>)	46
4.2.3	Condiciones meteorológicas (<code>estadisticas_meteorologicas.py</code>) . . .	47
4.2.4	Relaciones piloto-equipo-tiempo: mapas de calor (<code>heatmaps.py</code>)	47
4.2.5	Artefactos generados	49
4.3	Análisis de la Dificultad del Trazado	49
4.3.1	Corrección de escala en la geometría reconstruida	49
4.3.2	Detección de curvas (<code>Analisis_trazado/deteccion_curvas.py</code>) . . .	51
4.3.3	Mapa de dificultad del trazado (<code>Analisis_trazado/mapa_trazado.py</code>)	52
4.3.4	Relación entre dificultad geométrica y comportamiento real (<code>Analisis_- trazado/perfil_velocidad_curvas.py</code>)	54
4.3.5	Validación multi-circuito de la reconstrucción geométrica	57
4.3.6	Artefactos generados y limitaciones	59
4.4	Análisis de Variabilidad de la Velocidad	60
4.4.1	Motivación y adaptación al contexto de F1	60
4.4.2	Arquitectura del módulo	61
4.4.3	Métricas convencionales	62
4.4.4	Métricas de secuencialidad: MAGE, CONGA e IFV	62
4.4.5	Fenotipo: TB_{p25} y AS_{p75}	63
4.4.6	Diagrama de Poincaré	64
4.4.7	DFAint y DFAraw	65
4.4.8	Resultados sobre la sesión de referencia	65

4.4.9	Artefactos generados y limitaciones	68
4.5	Modelado ARIMA	69
4.5.1	Diseño experimental	69
4.5.2	Preparación de los datos	72
4.5.3	Variable exógena: Índice de Complejidad de Curva $s(t)$	73
4.5.4	Experimento 1: generalización espacial (trozo del circuito)	74
4.5.5	Experimento 2: generalización temporal (vueltas distintas)	76
4.5.6	Comparación y conclusiones	78
5	Conclusiones	81
5.1	Cumplimiento de los objetivos	81
5.2	Principales hallazgos	82
5.3	Limitaciones	83
5.4	Líneas futuras	84

Índice de Figuras

1.1	Logotipo Python [1]	18
1.2	Librerías Python	19
1.3	Logotipo PyCharm [8]	20
1.4	Logotipos de Overleaf y LaTeX	20
2.1	Racewatch circuito	24
2.2	Racewatch accidente	25
2.3	Paneles Racewatch	25
2.4	Paneles F1Insights	26
4.1	Comparativa de frecuencias de interpolación (1, 10 y 100 Hz) en la frenada más exigente de la sesión de referencia.	40
4.2	Distribución del tiempo de vuelta según el compuesto de neumático.	46
4.3	Perfil de velocidad a lo largo del trazado en la mejor vuelta de cada piloto.	47
4.4	Mapa de calor del ritmo de carrera (piloto $\times$ número de vuelta).	48
4.5	Correlación entre variables de telemetría (velocidad, acelerador, RPM y marcha).	49
4.6	Mapa de dificultad continuo del trazado, coloreado según el radio de curvatura local.	53
4.7	Curvas identificadas en el trazado de Monza, numeradas y coloreadas según su dificultad relativa.	54
4.8	Radio mínimo de curvatura frente a velocidad mínima real por curva ($r = 0.804$), coloreada por dificultad geométrica.	56
4.9	Reconstrucción geométrica en los dos circuitos donde el método generaliza correctamente.	58
4.10	Reconstrucción geométrica en Mónaco (caso problemático) y Singapur (excepción positiva a la hipótesis inicial).	59
4.11	Diagrama de Poincaré de la vuelta más rápida de la sesión de referencia, con la elipse de semiejes SD1/SD2.	64

4.12	Distribución, entre las 48 vueltas de la sesión de referencia, de seis métricas de variabilidad de velocidad.	66
4.13	DFAint frente a la vida del neumático ($r = -0.521$).	67
4.14	CV de velocidad por compuesto de neumático.	68
4.15	Correlogramas ACF y PACF de la velocidad (vuelta más rápida de la sesión de referencia, submuestreada a 5 Hz, 40 rezagos). La ACF decae de forma gradual, sin un corte abrupto; la PACF, tras el primer rezago, muestra varios valores decrecientes pero no despreciables (rezagos 2–5) antes de estabilizarse cerca de cero, en lugar del corte limpio de un ejemplo de manual.	71
4.16	Experimento 1: MAE y RMSE de cada orden ARIMAX candidato sobre el conjunto de validación.	75
4.17	Experimento 1: coeficiente de determinación R^2 de cada orden ARIMAX candidato sobre el conjunto de validación.	75
4.18	Experimento 1: predicción ARIMAX a un paso frente a la velocidad real en el tramo de test.	76
4.19	Experimento 2: MAE y RMSE de cada orden ARIMAX candidato sobre el conjunto de validación.	77
4.20	Experimento 2: coeficiente de determinación R^2 de cada orden ARIMAX candidato sobre el conjunto de validación.	77
4.21	Experimento 2: predicción ARIMAX a un paso frente a la velocidad real en el grupo de vueltas de test.	78
4.22	Comparación del error final (MAE, RMSE) y del R^2 en test entre ambos experimentos de generalización.	79

Índice de Tablas

3.1	Temporización del Trabajo Fin de Grado	32
4.1	Fila real de telemetría bruta devuelta por FastF1 (<code>lap.get_telemetry()</code>), sin limpiar ni interpolar.	35
4.2	Comparativa de frecuencias de interpolación sobre la vuelta de referencia.	40
4.3	Artefactos generados por el <i>pipeline</i> de extracción y limpieza.	43
4.4	Artefactos generados por el pipeline de análisis exploratorio.	50
4.5	Curvas detectadas en el trazado de referencia (GP de Italia 2023).	52
4.6	Perfil de velocidad real por curva (GP de Italia 2023, HAM), cruzando la segmentación geométrica con la telemetría interpolada.	55
4.7	Validación de la reconstrucción geométrica en cuatro circuitos de la temporada 2023.	58
4.8	Artefactos generados por el análisis de dificultad del trazado.	60
4.9	Correlación de Pearson de cuatro métricas de variabilidad (una por familia) con el tiempo de vuelta y la vida del neumático.	66
4.10	Resumen comparativo de los dos experimentos de generalización. Ambos modelos son $\text{ARIMAX}(p, d, q)$ con la dificultad del trazado $s(t)$ (Sección 4.5.3) como variable exógena.	78

1

Introducción

La disponibilidad masiva de datos y el avance de las técnicas de Inteligencia Artificial han transformado profundamente la forma en que se analizan y modelan sistemas complejos. En particular, el aprendizaje automático y el análisis de series temporales permiten estudiar la evolución de variables que dependen del tiempo, facilitando la detección de patrones y la realización de predicciones de distintas variables.

En el deporte de alto rendimiento, el uso de datos se ha convertido en una herramienta clave para el estudio objetivo del rendimiento. La Fórmula 1 representa uno de los entornos más avanzados desde el punto de vista tecnológico, ya que cada sesión de competición genera grandes volúmenes de información procedente de sistemas de telemetría y cronometraje. Variables como la velocidad, los tiempos por vuelta o el comportamiento del monoplaza a lo largo del trazado permiten analizar de forma detallada el rendimiento en pista.

El carácter secuencial de estos datos hace que las técnicas de series temporales resulten especialmente adecuadas para su análisis. Modelos estadísticos como ARIMA permiten describir y predecir la evolución de variables en el tiempo a partir de sus valores pasados, proporcionando una base sólida y fácilmente interpretable para el estudio de sistemas dinámicos.

En este contexto, el presente Trabajo Fin de Grado aborda el análisis de datos reales de Fórmula 1 obtenidos mediante la librería *FastF1*. A partir de la extracción, limpieza y análisis exploratorio de la información, se estudia la relación entre la velocidad del monoplaza y la dificultad del trazado del circuito, así como la aplicación de modelos predictivos basados en series temporales para la estimación de variables relacionadas con el rendimiento.

1.1 Motivación

La Fórmula 1 genera grandes volúmenes de datos en tiempo real, lo que constituye una oportunidad excepcional para aplicar técnicas avanzadas de análisis y modelado predictivo. Sin embargo, aunque el entorno profesional dispone de herramientas de alto nivel para la explotación de estos datos, los estudios académicos basados en información accesible públicamente siguen siendo limitados, especialmente en lo que respecta al modelado temporal de variables dinámicas.

Muchas de las aplicaciones más conocidas de la inteligencia artificial se han centrado en el procesamiento de imágenes o del lenguaje natural. No obstante, uno de los ámbitos con mayor proyección es el análisis de series temporales, presente en disciplinas como la meteorología, la economía, la ingeniería industrial o el análisis de sistemas en tiempo real. En estos contextos, el reto no solo consiste en predecir valores futuros, sino en comprender la estructura temporal del fenómeno estudiado.

La Fórmula 1 constituye un caso paradigmático, ya que cada monoplace incorpora numerosos sensores que capturan información de forma continua, como la velocidad, las revoluciones del motor, la posición en el trazado, los tiempos por sector o las fases de frenada y aceleración. Esta telemetría permite estudiar el sistema como un proceso dinámico altamente estructurado.

La motivación principal de este trabajo radica en explorar el potencial de modelos de series temporales interpretables aplicados a datos reales y reproducibles de Fórmula 1. Frente a enfoques basados exclusivamente en modelos de caja negra, se propone analizar el rendimiento mediante técnicas estadísticas que permitan no solo predecir, sino también comprender la evolución temporal de las variables estudiadas. De este modo, el trabajo combina rigor metodológico, aplicabilidad práctica y reproducibilidad, contribuyendo al análisis académico de un entorno tecnológico de alta complejidad.

1.2 Objetivos

1.2.1 Objetivo general

El objetivo principal de este Trabajo Fin de Grado es analizar y modelar datos reales de Fórmula 1 con el fin de predecir velocidades y tiempos de vuelta, utilizando técnicas de análisis de datos y modelos de series temporales.

1.2.2 Objetivos específicos

Para alcanzar el objetivo general, se plantean los siguientes objetivos específicos:

- Acceder, limpiar y organizar los datos de vueltas de la Fórmula 1 en un circuito concreto utilizando la biblioteca *FastF1*, prestando especial atención a los siguientes tipos de información:
 - **Telemetría:** velocidad, posición del acelerador, freno, revoluciones por minuto (RPM), marcha engranada y uso del sistema DRS.
 - **Datos de vuelta:** tiempos por vuelta y por sector, pilotos y escuderías.
 - **Información del circuito:** longitud total, sectores y curvas, con el objetivo de relacionar la velocidad con la dificultad del trazado.
 - **Condiciones de la sesión:** variables contextuales que puedan influir en el rendimiento y mejorar la capacidad predictiva de los modelos.
- Analizar el rendimiento de los pilotos y las escuderías en dicho trazado, extrayendo estadísticas relevantes y generando visualizaciones informativas, como el estudio de la dificultad asociada a cada curva del circuito.
- Estudiar la variabilidad de la velocidad a lo largo del tiempo y su posible relación con la dificultad del trazado.
- Implementar un modelo predictivo basado en series temporales, como ARIMA, para estimar la velocidad o la dificultad del circuito en un instante t a partir de valores históricos ($t - 1$, $t - 2$, etc.).
- Generar un informe completo que recoja el análisis realizado, los resultados obtenidos y las conclusiones derivadas del estudio.

1.3 Tecnologías y herramientas

A continuación, se presentan las herramientas empleadas en este proyecto, así como una breve justificación de su elección.

Python: Se ha utilizado Python como lenguaje principal de programación debido a su amplia implantación en el ámbito científico y del análisis de datos, así como por la disponibilidad de un ecosistema robusto de librerías especializadas que permiten realizar procesamiento numérico eficiente, manipulación estructurada de datos y modelado estadístico de manera integrada y reproducible.



Figura 1.1. Logotipo Python [1]

Las librerías de Python utilizadas en el desarrollo del proyecto han sido las siguientes:

- **FastF1:** librería de Python utilizada para el acceso a datos oficiales de competiciones de Fórmula 1. Esta permite obtener de forma estructurada información básica de dichas sesiones, como los tiempos empleados por cada piloto en completar una vuelta al circuito, los tiempos parciales en distintos tramos del mismo (sectores) y determinados datos de telemetría del vehículo, relacionados con variables como la velocidad o la aceleración. FastF1 incorpora además un sistema automático de descarga y almacenamiento en caché, lo que optimiza el acceso a los datos y garantiza la reproducibilidad de los análisis realizados a partir de competiciones reales.
- **pandas:** utilizada para la manipulación y análisis de los datos en forma de DataFrames. Esta librería permite realizar tareas de limpieza, filtrado y transformación de los datos de manera eficiente.
- **NumPy:** empleada como soporte para operaciones numéricas y cálculo científico, especialmente útil en el tratamiento de arrays y en el procesamiento de datos numéricos.
- **SciPy:** utilizada para la regularización de la telemetría a paso temporal constante mediante *splines* cúbicos y para la reconstrucción paramétrica 2D de la geometría del trazado (`scipy.interpolate`), pieza central de `Extraccion_limpieza` (Sección 4.1).
- **Matplotlib:** utilizada para la visualización de datos, permitiendo generar gráficos y representaciones que facilitan el análisis exploratorio y la interpretación de los resultados.
- **Seaborn:** biblioteca de visualización estadística construida sobre Matplotlib, utilizada para generar gráficos más avanzados y estéticamente optimizados. Ha sido empleada especial-

mente en el análisis exploratorio de datos para representar distribuciones, diagramas de caja y bigotes, mapas de calor y relaciones entre variables, facilitando la detección de patrones y tendencias en las series temporales analizadas.

- **Statsmodels:** empleada para la implementación de modelos de series temporales, en particular el modelo ARIMA, debido a su robustez y a la facilidad para evaluar el comportamiento y la precisión de los modelos estadísticos.



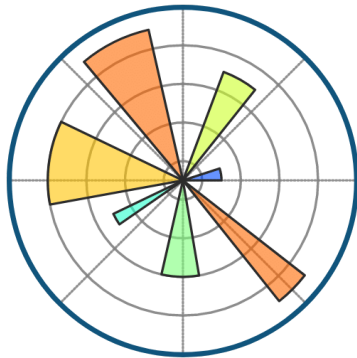
(a) Logotipo FastF1 [2]



(b) Logotipo pandas [3]



(c) Logotipo NumPy [4]



(d) Logotipo Matplotlib [5]



(e) Logotipo seaborn [6]



(f) Logotipo statsmodels [7]

Figura 1.2. Librerías Python

PyCharm: entorno de desarrollo integrado (IDE) utilizado para el desarrollo del proyecto, elegido por sus herramientas de depuración, gestión de entornos virtuales y facilidad para organizar proyectos de tamaño medio.



Figura 1.3. Logotipo PyCharm [8]

LaTeX: sistema de composición de textos empleado para la redacción de la memoria del Trabajo Fin de Grado, elegido por su capacidad para generar documentos estructurados y de alta calidad tipográfica.

Overleaf: plataforma online basada en LaTeX utilizada para la edición colaborativa y compilación del documento, que facilita la gestión del proyecto y el control de versiones de la memoria.



(a) Logotipo LaTeX [9]



(b) Logotipo Overleaf [10]

Figura 1.4. Logotipos de Overleaf y LaTeX

1.4 Estructura de la memoria

El resto de la memoria se organiza en cuatro capítulos. El Capítulo 2 (Estado del Arte) sitúa este TFG frente a las soluciones profesionales de análisis de datos en Fórmula 1 (RaceWatch, F1 Insights) y frente a las herramientas y trabajos académicos existentes sobre telemetría y predicción de rendimiento. El Capítulo 3 (Metodología de trabajo) describe la metodología ágil e iterativa seguida, los principios de diseño y organización del código, la fuente de los datos (FastF1) y la planificación temporal del proyecto. El Capítulo 4 (Implementación) constituye el núcleo técnico de la memoria y recoge, en orden, las cinco fases del pipeline: extracción y limpieza de los datos de sesión, análisis exploratorio, análisis de la dificultad geométrica del trazado –incluyendo su validación cruzada con telemetría real y en varios circuitos–, análisis de variabilidad de la velocidad mediante métricas adaptadas de la literatura de variabilidad glucémica y, por último, el modelado predictivo mediante ARIMA. Finalmente, el Capítulo 5 (Conclusiones) recoge los resultados obtenidos frente a los objetivos planteados en el

anteproyecto, las limitaciones identificadas a lo largo del desarrollo y las líneas de trabajo futuro que se derivan de ellas.

2

Estado del Arte

Las escuderías de Fórmula 1 diseñan de forma continua estrategias con el objetivo de maximizar el rendimiento de los monoplazas, apoyándose en la telemetría capturada durante cada sesión de carrera. A diferencia de los comienzos de la competición, donde el análisis se basaba en cronometraje manual y observación directa, en la actualidad cada punto del circuito recorrido por el vehículo queda registrado con gran precisión gracias a los más de 300 sensores integrados en el monoplaza, capaces de generar más de un millón de datos por segundo.

A partir de estos datos, las escuderías pueden ajustar parámetros como la presión y gestión de los neumáticos, la dureza de la suspensión o el reparto de frenada, con el fin de optimizar el comportamiento del vehículo en pista y anticiparse a situaciones perjudiciales, como el final de la vida útil de los neumáticos, el consumo crítico de combustible o la posible aparición del safety car.

El elevado volumen y la complejidad de la telemetría hacen imprescindible el uso de herramientas capaces de procesar, integrar y analizar esta información de manera eficiente. En este contexto, han surgido diversas plataformas y librerías especializadas en el análisis de datos de Fórmula 1, que permiten estructurar, visualizar y explotar la telemetría con fines analíticos y predictivos, constituyendo el punto de partida del presente estado del arte.

2.1 Soluciones profesionales

En el ámbito profesional, la Fórmula 1 emplea plataformas avanzadas de análisis y estrategia desarrolladas por empresas especializadas. Un ejemplo destacado es RaceWatch [11], una solución desarrollada en colaboración con la FIA y la empresa Catapult [12], utilizada en el entorno profesional para el procesamiento en tiempo real de datos de telemetría, cronometraje y eventos de carrera.

Paralelamente, la explotación de datos en la Fórmula 1 no se limita al entorno técnico de los equipos. Iniciativas como F1 Insights [13], desarrollada por la Formula 1 en colaboración con Amazon Web Services, trasladan el análisis avanzado de datos al ámbito de la retransmisión televisiva. A diferencia de plataformas orientadas a la toma de decisiones internas, F1 Insights está diseñada para enriquecer la experiencia del espectador mediante gráficos basados en inteligencia artificial y aprendizaje automático, que contextualizan en tiempo real estrategias, comparativas de rendimiento y situaciones clave de carrera.

2.1.1 Racewatch

RaceWatch es una plataforma en constante evolución que integra múltiples fuentes de información en una única interfaz. Entre sus principales funcionalidades destacan las siguientes:

- Integración de datos de cronometraje, telemetría, GPS, comunicaciones y vídeo en una única vista operativa, facilitando el seguimiento completo de la carrera. En la Figura 2.1 se observa cómo se representan mediante colores las diferencias de rendimiento por sectores del circuito.

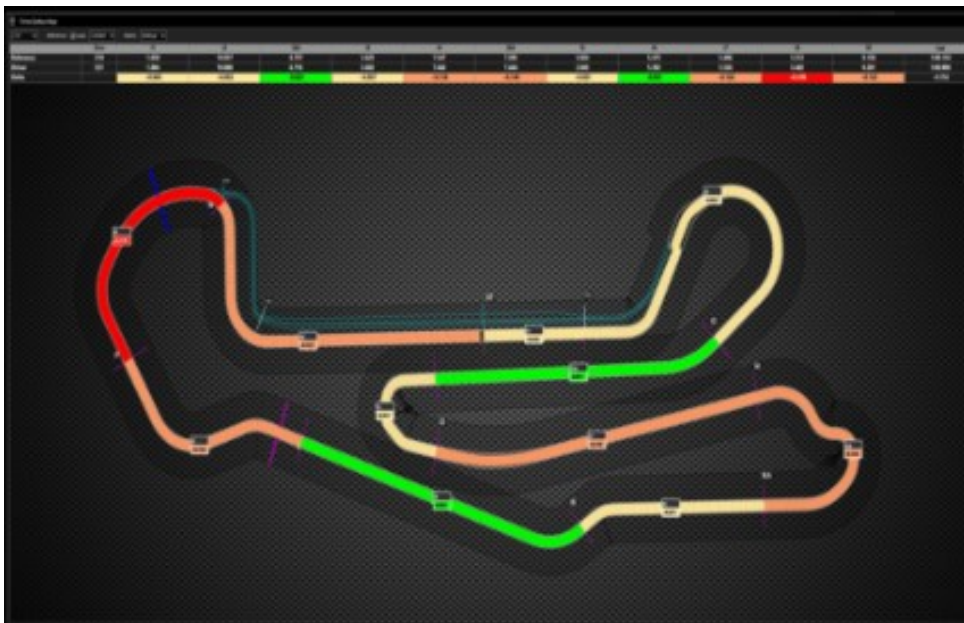


Figura 2.1. Racewatch circuito

- Seguimiento en tiempo real del comportamiento de cada monoplaza mediante modelos que comparan el rendimiento observado con patrones esperados. Esta capacidad permite detectar anomalías y situaciones relevantes de forma inmediata.

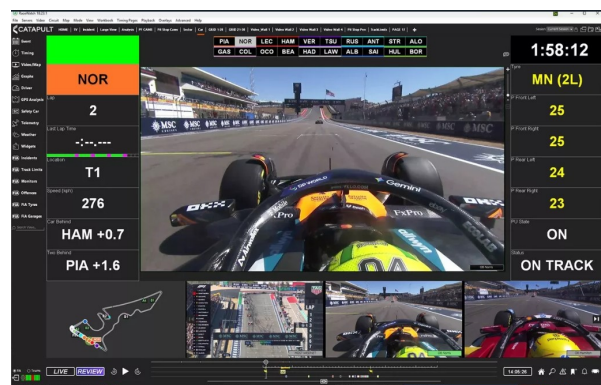


Figura 2.2. Racewatch accidente

- Configuración de paneles personalizados adaptados a distintos entornos operativos, como el muro de boxes o los centros remotos de análisis, permitiendo una visualización flexible y compartida de la información:



(a) Racewatch panel 1



(b) Racewatch panel 2

Figura 2.3. Paneles Racewatch

2.1.2 F1 Insights

La plataforma se apoya en la gran cantidad de datos generados por los monoplazas, combinando telemetría en tiempo real con información histórica acumulada durante décadas. Estos datos son procesados mediante infraestructuras en la nube proporcionadas por Amazon Web Services y modelos

predictivos que permiten estimar la evolución de la carrera, luego no se limita a mostrar estadísticas descriptivas, sino que incorpora modelos que proyectan escenarios futuros basados en la evolución temporal del ritmo de carrera y en patrones históricos. De este modo, el espectador puede interpretar cómo variaciones en el punto de frenada, la gestión de neumáticos o el ritmo por vuelta influyen en el desarrollo de la competición. Este enfoque evidencia el potencial del análisis de datos masivos y de las técnicas de inteligencia artificial aplicadas a entornos deportivos.

Entre sus funcionalidades más relevantes se encuentran la predicción de estrategias de parada en boxes, la estimación del tiempo perdido tras errores de conducción, el análisis detallado del rendimiento en curvas y la proyección de posibles adelantamientos (Figura 2.4).



(a) F1Insights panel 1

(b) F1Insights panel 2

Figura 2.4. Paneles F1Insights

2.2 Soluciones académicas

En contraste con estas soluciones profesionales, existen herramientas públicas que permiten el análisis de datos reales de Fórmula 1 en contextos académicos y de investigación. FastF1 [14], la librería empleada en este TFG (Sección 3.2), es la única de ellas que expone telemetría de alta frecuencia –velocidad, acelerador, freno, marcha, posición– vuelta a vuelta y no solo resultados agregados, lo que la hace imprescindible para el tipo de análisis planteado aquí. Ergast [15] fue durante más de una década la fuente de referencia para datos históricos de resultados y cronometrage mediante una API REST pública, pero el proyecto se discontinuó a finales de 2024 por falta de financiación y de un plan de sostenibilidad a largo plazo, siendo sustituido por iniciativas comunitarias como Jolpica-F1; se cita aquí por su relevancia histórica en el ecosistema, no como herramienta actualmente operativa. OpenF1 [16] ha tomado en buena medida su lugar desde 2023, ofreciendo datos en tiempo real y

ampliando progresivamente su cobertura histórica. Asimismo, han surgido dashboards interactivos que muestran la telemetría de forma visual e intuitiva, como f1-dash [17] o f1-tempo [18], orientados principalmente a la exploración y comparación de datos. No obstante, la mayoría de estas herramientas se centran en el análisis descriptivo y en la visualización de información, siendo menos frecuentes los trabajos que integran modelos predictivos aplicados directamente sobre datos de telemetría accesibles públicamente. En particular, son escasos los estudios que abordan la predicción de variables dinámicas como la velocidad o el tiempo por vuelta mediante modelos interpretables.

Se ha encontrado un trabajo académico reciente que aplica técnicas avanzadas de aprendizaje automático a datos históricos de telemetría de Fórmula 1 [19], publicado en IEEE INDISCON 2025. En dicho estudio se desarrollan modelos de aprendizaje profundo y XGBoost para predecir la energía aplicada a los neumáticos durante la carrera, con el objetivo de estimar su degradación y optimizar decisiones estratégicas como el momento de parada en boxes. Asimismo, incorpora técnicas de inteligencia artificial para interpretar el comportamiento de los modelos predictivos.

De manera complementaria, otro trabajo reciente aborda la predicción de tiempos por vuelta y resultados de carrera en Fórmula 1 mediante técnicas de aprendizaje automático aplicadas a datos históricos comprendidos entre 2014 y 2023 [20] —un proyecto de fin de grado (*capstone project*) en Computer Science de la New York University Abu Dhabi, y no un artículo revisado por pares, aunque se incluye aquí por la cercanía directa de su planteamiento con el de este TFG—. En este estudio se realiza inicialmente una comparación entre modelos tradicionales como regresión lineal, árboles de decisión y Random Forest, estableciendo una línea base de rendimiento. Posteriormente, se implementan redes neuronales recurrentes (LSTM) diseñadas para capturar dependencias temporales en datos secuenciales como los tiempos por vuelta.

2.3 Fundamentos metodológicos

Además de las herramientas específicas del dominio de la Fórmula 1, este TFG se apoya en dos cuerpos de literatura metodológica de propósito general, ajenos al automovilismo, que se detallan técnicamente en el Capítulo 4 y se introducen aquí brevemente para situarlos en su contexto.

En primer lugar, el modelado predictivo de la velocidad (Sección 4.5) se basa en la metodología clásica de Box y Jenkins para el análisis de series temporales [21], que da lugar a los modelos ARIMA (*AutoRegressive Integrated Moving Average*) y a su extensión con variables exógenas, ARIMAX [22].

Frente a los modelos de aprendizaje profundo empleados en los dos trabajos académicos citados en la sección anterior, ARIMA representa el extremo opuesto del espectro de complejidad: un modelo lineal, con pocos parámetros, cuya principal ventaja es la interpretabilidad directa de sus coeficientes y su bajo requisito de datos de entrenamiento, a costa de una capacidad más limitada para capturar relaciones no lineales. El uso de la geometría del trazado como variable exógena para modular una predicción de rendimiento —el papel que aquí desempeña el Índice de Complejidad de Curva $s(t)$ (Sección 4.5.3)— conecta además con trabajos de simulación de tiempo de vuelta basados en la curvatura del circuito, como el de Doyle et al. [23].

En segundo lugar, el análisis de variabilidad de la velocidad (Sección 4.4) adapta un conjunto de métricas desarrolladas originalmente para cuantificar la variabilidad de la glucemia en la monitorización continua de pacientes diabéticos (*Continuous Glucose Monitoring*), en particular el trabajo de Colás et al. [24], tomando como referencia de implementación un repositorio de análisis de variabilidad glucémica de código abierto [25]. Esta transferencia metodológica entre dominios —de la fisiología a la telemetría de un vehículo de competición— no es una analogía superficial: ambos escenarios comparten la misma estructura estadística subyacente, una serie temporal fisiológica o cinemática que fluctúa alrededor de una tendencia, y cuya variabilidad, no solo su nivel medio, resulta clínicamente o técnicamente informativa. No se ha encontrado en la literatura de análisis deportivo de Fórmula 1 revisada un precedente directo de esta adaptación.

En este contexto se sitúa el presente Trabajo Fin de Grado, cuyo objetivo no es únicamente visualizar o comparar datos históricos, sino desarrollar un enfoque predictivo basado en técnicas estadísticas aplicadas a telemetría real. A diferencia de [19] y [20], que recurren a modelos de aprendizaje profundo (XGBoost y LSTM, respectivamente) entrenados sobre varias temporadas de datos históricos, este trabajo opta deliberadamente por un modelo estadístico clásico —ARIMA, extendido a ARIMAX—, interpretable por construcción (sus coeficientes se corresponden directamente con la dependencia de la velocidad respecto a sus propios valores pasados y respecto a la dificultad geométrica del trazado) y que no requiere el volumen de datos de entrenamiento que exigen las redes neuronales, a cambio de una capacidad más limitada para capturar relaciones no lineales complejas. De esta forma, se pretende contribuir al análisis académico del rendimiento en Fórmula 1 mediante un modelo reproducible que permita estimar la evolución temporal de determinadas variables clave del monoplaza.

3

Metodología de trabajo

El desarrollo del presente Trabajo Fin de Grado se ha abordado siguiendo una metodología incremental e iterativa, inspirada en los principios de las metodologías ágiles de desarrollo software. Este enfoque ha permitido estructurar el trabajo en ciclos sucesivos de desarrollo (sprints), cada uno centrado en una fase concreta del proyecto, tales como la extracción y preparación de los datos, el análisis exploratorio, el modelado mediante técnicas de series temporales y la evaluación del modelo predictivo.

A diferencia de un enfoque secuencial clásico, la aproximación iterativa ha facilitado la revisión continua de los resultados intermedios y la introducción de mejoras progresivas en cada etapa. Este planteamiento ha resultado especialmente adecuado en un contexto de análisis de datos temporales, donde decisiones como la selección de variables relevantes, la transformación de las series o el ajuste de parámetros del modelo requieren validación experimental constante. De este modo, cada iteración ha permitido detectar posibles limitaciones, reformular hipótesis de trabajo y refinar el sistema antes de avanzar hacia fases posteriores.

Cada sprint ha generado entregables intermedios, que han servido como base para la toma de decisiones técnicas en etapas posteriores. Estos avances han sido revisados periódicamente por el tutor, Miguel Ángel Molina Cabello, y el co-tutor, Ezequiel López Rubio, garantizando la coherencia del desarrollo con los objetivos definidos en el anteproyecto y asegurando un seguimiento continuo del progreso.

Desde el punto de vista científico, el trabajo se apoya asimismo en el método científico aplicado al análisis de datos. Este se ha estructurado en las siguientes etapas: formulación del problema y definición de objetivos, obtención y preparación de los datos, análisis exploratorio y detección de patrones, planteamiento e implementación del modelo predictivo, experimentación controlada y evaluación cuantitativa de resultados, y finalmente, interpretación de los mismos y extracción de conclusiones. Este

enfoque sistemático garantiza la coherencia metodológica, la trazabilidad de las decisiones adoptadas y la reproducibilidad del estudio realizado.

3.1 Diseño y organización del código

Durante el desarrollo se ha prestado especial atención a la arquitectura del sistema, aplicando principios de modularidad, separación de responsabilidades, extensibilidad y mantenibilidad. La estructura del proyecto permite diferenciar claramente las distintas fases del proceso analítico, facilitando futuras ampliaciones o modificaciones sin comprometer la estabilidad del sistema.

El proyecto se ha organizado siguiendo una estructura jerárquica que distingue entre datos brutos, datos procesados, scripts de análisis y modelado, así como resultados experimentales y figuras generadas. Esta separación favorece la reutilización de componentes, la replicabilidad de los experimentos y la incorporación futura de nuevos modelos predictivos o nuevas fuentes de información.

Asimismo, el código se ha estructurado en módulos independientes, cada uno responsable de una funcionalidad específica, lo que reduce el acoplamiento entre componentes y mejora la claridad del diseño. Esta organización permite reutilizar partes del sistema en otros contextos de investigación sin necesidad de realizar modificaciones significativas en la arquitectura general.

En concreto, cada módulo del proyecto (`Extraccion_limpieza`, `Analisis_exploratorio`, `Analisis_trazado`, `Analisis_variabilidad` y `Modelado_arima`) expone un único punto de entrada ejecutable, `main.py`. Cuando un módulo necesita realizar varias tareas relacionadas pero distintas –por ejemplo, `Extraccion_limpieza` extrae la sesión de referencia, compara frecuencias de interpolación y valida la reconstrucción geométrica en otros circuitos– estas se exponen como modos seleccionables mediante el argumento `--modo`, en lugar de multiplicar scripts ejecutables independientes dentro de la misma carpeta. Por la misma razón, ningún módulo importa código directamente de otro: la comunicación entre fases del proyecto se realiza exclusivamente a través de artefactos CSV persistidos en disco, de modo que cada módulo se pueda ejecutar, probar y entender de forma aislada. Todos los artefactos generados por `Extraccion_limpieza` –incluidos los específicos para `Modelado_arima` (Sección 4.5.2) o para la validación multi-circuito (Sección 4.3.5)– se guardan en `csv_data/`, organizados en subcarpetas según su propósito (`csv_data/arima/`, `csv_data/multicircuito/`, `csv_data/comparar_frecuencias/`). Las tablas resumen y figuras que sí produce cada módulo de análisis posterior –incluido `Modelado_arima`, pese a ser el único que

consume artefactos de `csv_data/arima/` en lugar de `csv_data/` directamente— se organizan de manera uniforme en directorios de nivel superior dedicados a cada tipo de salida (`eda/` y `figures/`), subdivididos a su vez en una subcarpeta por módulo generador (p. ej. `eda/analisis_exploratorio/`, `eda/modelado_arima/` o `figures/analisis_trazado/`).

Con el objetivo de garantizar la reproducibilidad de los experimentos, los distintos escenarios de prueba se han configurado mediante parámetros externos, evitando modificaciones manuales del código fuente. Este enfoque facilita la ejecución sistemática de distintas configuraciones experimentales y mejora la comparabilidad de los resultados obtenidos.

Desde el punto de vista del control del desarrollo, se ha utilizado Git como sistema de control de versiones y GitHub como plataforma de alojamiento del repositorio. Este mecanismo permite mantener un historial detallado de cambios, documentar hitos relevantes del proyecto y recuperar versiones anteriores en caso necesario, garantizando así la trazabilidad completa del proceso de implementación.

Finalmente, el código ha sido documentado mediante comentarios descriptivos y el uso de una nomenclatura coherente en variables y funciones, siguiendo buenas prácticas de ingeniería del software. Este esfuerzo facilita tanto el mantenimiento del sistema como su comprensión por parte de terceros.

3.2 Fuente de los datos

Los datos empleados en este trabajo se han obtenido a través de la librería `FastF1`, un paquete de Python de código abierto (licencia MIT) que permite descargar, procesar y analizar datos oficiales de Fórmula 1.

`FastF1` accede a la información pública proporcionada por el sistema oficial de *Live Timing* de la competición, incluyendo datos de telemetría, cronometraje y rendimiento de pilotos y escuderías. Estos datos están disponibles para fines de investigación, docencia y análisis no comercial, lo que garantiza tanto su accesibilidad como la adecuación legal de su utilización en el contexto del presente Trabajo Fin de Grado.

La librería incorpora además mecanismos de descarga automática y almacenamiento en caché local, lo que optimiza el acceso repetido a la información y reduce la dependencia de consultas externas. Esta característica resulta especialmente relevante cuando se trabaja con grandes volúmenes de datos temporales procedentes de sesiones completas de competición, ya que mejora la eficiencia del

procesamiento y contribuye a la reproducibilidad de los análisis realizados.

3.3 Planificación

La Tabla 3.1 recoge la temporización estimada del Trabajo Fin de Grado, distribuida en un conjunto de fases que cubren el ciclo completo de un proyecto de análisis de datos: revisión bibliográfica, adquisición y preparación de datos, análisis exploratorio, modelado, evaluación y redacción final. La asignación de horas se ha realizado buscando un equilibrio entre tareas de investigación y tareas de implementación, priorizando aquellas etapas con mayor carga técnica (análisis exploratorio, modelado y redacción). Esta planificación ha servido como guía para organizar el trabajo de forma incremental, permitiendo definir entregables intermedios y revisar el progreso de manera periódica.

FASE	HORAS
Estado del arte	15
Estudio de FastF1 y APIs	25
Extracción y limpieza de datos	30
Análisis exploratorio	40
Análisis de la dificultad del trazado	30
Presentación visual de resultados	30
Modelado ARIMA	40
Evaluación de ARIMA	26
Conclusiones y recomendaciones	15
Redacción de memoria	45
TOTAL	296

Tabla 3.1. Temporización del Trabajo Fin de Grado

4

Implementación

Como ya se mencionó anteriormente, usaremos la librería `FastF1`. A partir de ella, se realiza la limpieza y extracción de sus datos, luego se realiza el análisis exploratorio.

4.1 Extracción y limpieza de datos

4.1.1 Introducción y objetivos

Este apartado describe el módulo `Extraccion_limpieza`, encargado de la primera fase del proyecto: la obtención de datos brutos de sesiones de Fórmula 1 y su transformación en un conjunto de artefactos tabulares limpios, homogéneos y trazables que sirvan de entrada a las fases posteriores de análisis e ingeniería de características (interpolación, cálculo de curvatura del trazado, etc.).

Los datos se obtienen a través de la librería `FastF1`, que proporciona acceso a resultados oficiales de sesión, tiempos por vuelta y telemetría de alta frecuencia (posición, velocidad, acelerador, freno, marcha, DRS, ...) capturada por los sistemas de cronometraje de la F1. El módulo se organiza como un *pipeline* secuencial de extracción – limpieza – persistencia, en el que cada etapa se encapsula en un fichero Python independiente con una responsabilidad única, y se orquesta desde un script principal (`main.py`).

Los objetivos concretos de esta fase son:

- Cargar una sesión de F1 concreta (año, Gran Premio y tipo de sesión) de forma reproducible, aprovechando un sistema de caché local.
- Depurar los tiempos de vuelta, descartando entradas/salidas de *pit lane* y vueltas atípicas.

- Extraer la telemetría de alta frecuencia de las vueltas más representativas (las mejores de cada piloto, o las de un piloto concreto) y limpiarla de valores inválidos.
- Extraer y depurar las condiciones meteorológicas de la sesión.
- Estimar la longitud y reconstruir la geometría 2D del trazado a partir de la telemetría, calculando la curvatura de la pista.
- Regularizar temporalmente la telemetría mediante interpolación por *splines* cúbicos, para obtener series a paso constante aptas para modelos de series temporales (ARIMA).
- Persistir todos los artefactos resultantes en formato CSV, junto con metadatos de trazabilidad (año, Gran Premio, sesión).

4.1.2 Información proporcionada por FastF1

Antes de describir la arquitectura del pipeline y las transformaciones de limpieza aplicadas, conviene precisar qué información bruta expone *FastF1* para cada sesión, ya que de ella dependen tanto las decisiones de depuración descritas más adelante como las limitaciones del conjunto de datos resultante.

Para cada vuelta completada, la API devuelve, entre otros, los siguientes campos:

- **Identificación y resultado:** piloto (`Driver`), equipo (`Team`), número de vuelta (`LapNumber`), tiempo total (`LapTime`) y tiempos parciales por sector (`Sector1Time`, `Sector2Time`, `Sector3Time`).
- **Neumáticos:** compuesto utilizado (`Compound`: blando, medio o duro, ordenados de mayor a menor rendimiento y de menor a mayor durabilidad), vueltas recorridas con ese juego (`TyreLife`) y si es un juego nuevo (`FreshTyre`).
- **Pit lane:** instantes de entrada y salida de boxes (`PitInTime`, `PitOutTime`), empleados en la limpieza para descartar vueltas no representativas del ritmo en pista.

Para cada vuelta puede solicitarse además su telemetría de alta frecuencia, con los canales:

- **Time y Distance:** instante dentro de la vuelta y distancia recorrida desde su inicio.
- **X, Y:** posición del monoplaza en el plano del circuito, expresada en decímetros y no en metros como el resto de canales (Sección 4.3 detalla la corrección de escala aplicada antes de usarlas para reconstruir la geometría del trazado).

- **Speed**: velocidad instantánea, en km/h.
- **Throttle, Brake**: porcentaje de acelerador aplicado y estado del freno.
- **nGear, RPM**: marcha engranada y régimen de motor.
- **DRS**: estado del sistema DRS¹.

A nivel de sesión completa, FastF1 proporciona también una serie de condiciones meteorológicas muestreada aproximadamente cada minuto, con los campos **AirTemp**, **Humidity**, **Pressure**, **Rainfall**, **TrackTemp**, **WindDirection** y **WindSpeed**.

La Tabla 4.1 muestra una fila real de telemetría bruta, sin procesar, tal como la devuelve la API para la primera muestra de la vuelta de referencia (HAM, vuelta 29, GP de Italia 2023) – previa, por tanto, a cualquiera de las transformaciones descritas en las secciones siguientes.

Tabla 4.1. Fila real de telemetría bruta devuelta por FastF1 (`lap.get_telemetry()`), sin limpiar ni interpolar.

Time	Distance (m)	X	Y	Speed (km/h)	
00:00:00.000	0.01	−1371.72	−651.89	315.2	
	Throttle (%)	Brake	RPM	nGear	DRS
	100.0	False	11615.8	8	8

No toda la información necesaria para este trabajo está disponible directamente en la API: en particular, la longitud del circuito y su geometría 2D reconstruida –imprescindibles para el análisis de dificultad del trazado (Sección 4.3)– no forman parte de la respuesta de FastF1 y se estiman a partir de la propia telemetría de posición, como se detalla en la Sección 4.1.8.

4.1.3 Arquitectura del pipeline

El módulo se compone de seis ficheros con responsabilidades bien diferenciadas, más un script orquestador:

¹Sistema aerodinámico que abre una parte del alerón trasero para reducir la resistencia y ganar velocidad en recta. Solo puede activarse en zonas homologadas por la FIA y, en carrera, únicamente si el piloto está a menos de un segundo del coche que le precede.

- `sesion.py` – carga de la sesión de FastF1 y gestión de la caché local.
- `vueltas.py` – limpieza y filtrado de los tiempos de vuelta.
- `telemetria.py` – extracción, limpieza y regularización (temporal y geométrica) de la telemetría.
- `condiciones_meteorologicas.py` – extracción y limpieza de los datos meteorológicos.
- `circuito.py` – estimación de la longitud del circuito.
- `almacenar_csv.py` – persistencia de DataFrames en disco.
- `main.py` – único punto de entrada ejecutable del módulo, seleccionable mediante el argumento `--modo`: el pipeline completo (`pipeline`, por defecto), la comparativa de frecuencias de interpolación (`comparar-frecuencias`, página 39) o la validación multi-circuito (`multicircuito`, Sección 4.3.5).

El flujo de ejecución definido en `main.py --modo pipeline` (el modo por defecto) sigue los siguientes pasos, para una sesión configurada mediante los argumentos de línea de comandos `--year`, `--gp`, `--session` y `--driver` (gestionados con `argparse`, con valores por defecto que reproducen la configuración original: temporada 2023, Gran Premio de Italia, sesión de carrera "R" y piloto de referencia "HAM"), lo que permite reejecutar el pipeline sobre otra sesión sin modificar el código:

1. Carga de la sesión con `cargar_sesion`.
2. Limpieza de vueltas con `limpiar_vueltas` y exportación a `laps_clean.csv`.
3. Extracción de telemetría de las tres mejores vueltas de cada piloto y de las mejores vueltas de un piloto concreto, ambas mediante la función `extraer_telemetria` (parametrizada por el argumento `drivers`), con su correspondiente limpieza (`limpiar_telemetria`) y exportación a `telemetry_3_mejores.csv` y `telemetry_piloto.csv`.
4. Extracción y limpieza de las condiciones meteorológicas, exportadas a `weather.csv`.
5. Estimación de la longitud del circuito (`estimar_longitud_circuito`).
6. Regularización dual de la telemetría del piloto seleccionado:
 - (a) Interpolación temporal 1D vuelta a vuelta a 100 Hz (`interpoliar_telemetria_temporal`), exportada a `telemetry_piloto_interpolada.csv`.

(b) Reconstrucción paramétrica 2D del trazado a partir de la vuelta más rápida (`interpolate_geometry_y_telemetria_circuito`), exportada a `curvatura_interpolada.csv`.

7. Persistencia de metadatos de control en `circuit_info.txt` (año, Gran Premio, sesión y longitud estimada del circuito).

Todos los artefactos se almacenan en el directorio `csv_data/`, creado automáticamente si no existe.

4.1.4 Carga de la sesión: `session.py`

La función `cargar_sesion(year, gp, session_type, cache_dir=None)` encapsula el acceso a la API de FastF1. Antes de solicitar los datos, habilita un sistema de caché local (`fastf1.Cache.enable_cache`) ubicado, por defecto, en un directorio `cache/` relativo a la propia ubicación del módulo (y no al directorio de trabajo desde el que se invoque el script), evitando así descargas repetidas y garantizando la reproducibilidad de las ejecuciones. A continuación obtiene el objeto de sesión con `fastf1.get_session(year, gp, session_type)` y fuerza su carga en memoria mediante `session.load()`, paso indispensable para que los tiempos de vuelta, la telemetría y los datos meteorológicos queden disponibles para el resto del pipeline.

4.1.5 Limpieza de tiempos de vuelta: `vueltas.py`

La función `limpiar_vueltas` parte del `DataFrame` de vueltas de la sesión (`session.laps`) y aplica la siguiente secuencia de transformaciones:

1. **Selección de columnas relevantes:** se conservan únicamente las columnas necesarias para el análisis (piloto, equipo, número de vuelta, tiempos de vuelta y de sector, compuesto de neumático, vida del neumático, y marcas de entrada/salida de *pit lane*), comprobando su existencia previamente para tolerar variaciones del esquema devuelto por FastF1.
2. **Trazabilidad:** se añaden las columnas `Year`, `GrandPrix` y `Session` para identificar el origen de cada fila una vez los datos de distintas sesiones se combinen aguas abajo.
3. **Eliminación de vueltas de *pit lane*:** se descartan las vueltas con marca de entrada o salida de *pit lane* (`PitInTime` / `PitOutTime` no nulos), ya que estas vueltas no son representativas del rendimiento en pista.

4. **Eliminación de tiempos nulos:** se descartan las filas sin LapTime registrado.
5. **Conversión de unidades:** el tiempo de vuelta, representado internamente como Timedelta, se convierte a segundos en la columna LapTime_s.
6. **Filtrado de valores atípicos:** se descarta el 1 % de vueltas más lentas, calculando el percentil 99 de LapTime_s y conservando únicamente las vueltas por debajo de dicho umbral. Con ello se eliminan vueltas afectadas por incidencias (banderas amarillas, tráfico, errores de pilotaje) sin necesidad de reglas ad-hoc.

4.1.6 Extracción y limpieza de telemetría: `telemetry.py`

Este es el fichero de mayor complejidad del módulo, ya que combina la extracción de series de alta frecuencia con dos estrategias de regularización distintas.

4.1.6.1 Extracción

La extracción se realiza mediante una única función, `extraer_telemetria(session, year, gp, session_type, drivers=None, max_laps_per_driver=3)`, basada en `session.laps.pick_quicklaps()` para descartar de partida vueltas lentas no representativas (*safety car*, *pit stops*, etc.). El parámetro `drivers` determina su alcance: si es `None` se procesan las mejores vueltas de *todos* los pilotos de la sesión; si se pasa un piloto concreto (cadena de texto) o una lista de pilotos, la extracción se restringe a ellos. De este modo, una misma función cubre tanto la extracción de las mejores vueltas de cada piloto como el aislamiento del comportamiento de un piloto de referencia (en la configuración actual, Lewis Hamilton, "HAM"), evitando la duplicación de lógica que existía originalmente entre dos funciones casi idénticas. Cuando se solicitan pilotos concretos que no tienen vueltas válidas en la sesión, la función lo notifica por consola y devuelve un `DataFrame` vacío en lugar de fallar.

Para cada piloto seleccionado se toman sus `max_laps_per_driver` vueltas más rápidas y se obtiene la telemetría asociada a cada una mediante `lap.get_telemetry()`. Por cada vuelta procesada se conservan únicamente los canales de interés (`Time`, `Distance`, `X`, `Y`, `Speed`, `Throttle`, `Brake`, `RPM`, `nGear`, `DRS`), filtrando de nuevo por columnas existentes para tolerar variaciones de esquema, y se enriquece cada fila con metadatos de trazabilidad (`LapID`, `Year`, `GrandPrix`, `Session`, `Driver`, `LapNumber`, `LapTime_s`, `Time_s`). Los errores de extracción de una vuelta concreta (por ejemplo, telemetría no disponible) se capturan y se registran como aviso sin interrumpir el procesamiento del resto de vueltas, priorizando la robustez del pipeline frente a datos incompletos propios

de una API externa.

4.1.6.2 Limpieza

La función `limpiar_telemetria` aplica una depuración mínima: elimina las filas sin valor de `Speed` y las que presentan valores de velocidad negativos, que corresponden a artefactos de sensor sin sentido físico.

4.1.6.3 Regularización temporal 1D: `interpoliar_telemetria_temporal`

Las muestras de telemetría de FastF1 no llegan a paso temporal constante, lo que dificulta su uso directo en modelos de series temporales. Para resolver esto, `interpoliar_telemetria_temporal` regulariza la telemetría de una vuelta al paso deseado (por defecto, 0.01 s, es decir, 100 Hz) mediante `scipy.interpolate` [26], siguiendo los pasos:

1. Se ordena la vuelta por `Time_s` y se eliminan duplicados temporales, requisito de `scipy.interpolate.CubicSpline`.
2. Si la vuelta cuenta con menos de 4 puntos válidos (mínimo exigido por el spline cúbico), se devuelve sin regularizar, evitando errores en tramos con datos insuficientes.
3. Se genera una rejilla temporal homogénea entre el instante mínimo y máximo de la vuelta, con paso `frecuencia_s`.
4. Se ajusta un `CubicSpline` independiente para cada canal disponible (`Speed`, `Distance`, `Throttle`, `Brake`) en función de `Time_s`, y se evalúa sobre la nueva rejilla temporal.

El resultado es un `DataFrame` con muestras equiespaciadas en el tiempo, apto para su uso en modelos autorregresivos (ARIMA). En `main.py` este proceso se aplica vuelta a vuelta (agrupando por `LapID`) sobre la telemetría del piloto seleccionado, y los resultados se concatenan en `telemetry_piloto_interpolada.csv`.

Elección de la frecuencia de interpolación.

Se planteó si 1 s de paso temporal —el orden de magnitud descrito inicialmente en una presentación del avance del trabajo— sería suficiente, frente a alternativas más finas como 0.1 s o 0.01 s. Para responder a esta pregunta con datos, se implementó una comparación (`Extraccion_limpieza/main.py --modo comparar-frecuencias`) que interpola la misma vuelta de referencia a las tres frecuencias y mide, sobre la zona de frenada más exigente de la sesión (la variante del Rettifilo, Sección 4.3.4), cuántas muestras caen dentro de una frenada típica de 1.5 s (Tabla 4.2).

Tabla 4.2. Comparativa de frecuencias de interpolación sobre la vuelta de referencia.

Paso (s)	Frecuencia (Hz)	Dist. media entre muestras (m)	Muestras en una frenada de 1.5 s
1.0	1	67.25	2
0.1	10	6.73	15
0.01	100	0.67	150

A 1 Hz, la distancia media entre dos muestras consecutivas (67 m) supera la longitud de varias de las curvas del propio trazado (Tabla 4.5), y una frenada completa queda representada por apenas 2 puntos –insuficiente para captar su forma–, frente a los 15 puntos de 0.1 s o los 150 de 0.01 s. La Figura 4.1 confirma esto visualmente: a 1 Hz (rojo) la interpolación aproxima la frenada mediante segmentos rectos que ignoran por completo el pequeño repunte de velocidad hacia los 925 m, perfectamente capturado tanto a 0.1 Hz como a 0.01 Hz (10 y 100 Hz, prácticamente indistinguibles entre sí y de la propia telemetría bruta). Esto confirma que 0.01 s (100 Hz), el valor efectivamente empleado en el *pipeline*, es una elección adecuada –y no excesiva, dado el coste computacional moderado del ajuste de *splines* cúbicos– frente a alternativas más gruesas.

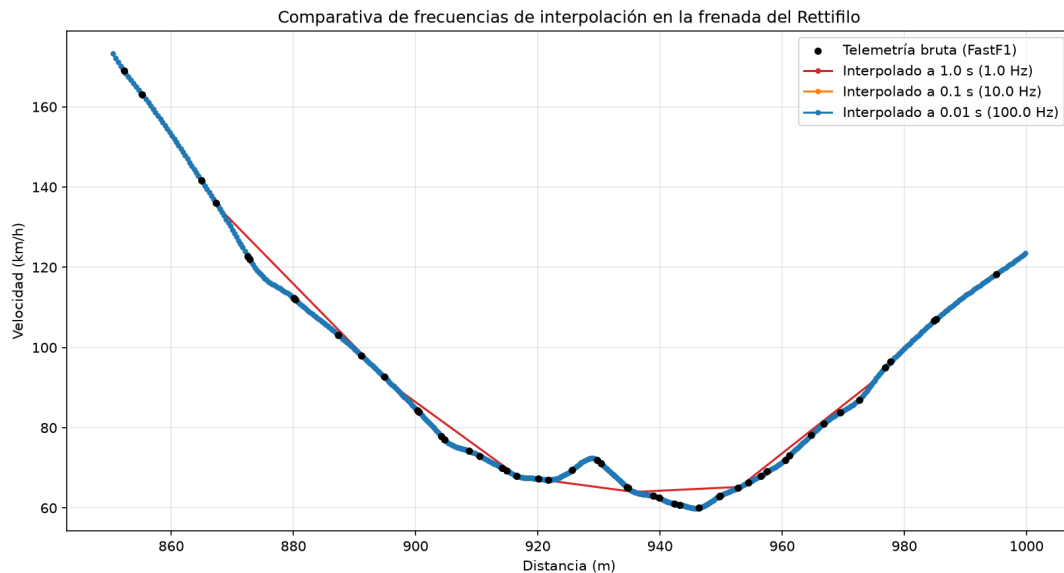


Figura 4.1. Comparativa de frecuencias de interpolación (1, 10 y 100 Hz) en la frenada más exigente de la sesión de referencia.

4.1.6.4 Reconstrucción geométrica 2D del trazado: `interpolar_geometria_y_telemetria_circuito`

A diferencia de la regularización anterior –centrada en el dominio temporal–, esta función reconstruye la geometría espacial del circuito mediante un *spline* paramétrico bidimensional, usando `scipy.interpolate.splprep` / `splev` sobre la vuelta de referencia (la vuelta más rápida del piloto seleccionado). El procedimiento es el siguiente:

1. La telemetría de la vuelta de referencia se ordena por `Distance` para garantizar coherencia espacial.
2. Se ajusta una curva paramétrica cerrada $(x(u), y(u))$ sobre las coordenadas (X, Y) mediante `splprep`, con parámetro de suavizado `smoothing` (80.0 por defecto), grado cúbico ($k = 3$) y periodicidad (`per=True`), adecuada para un trazado cerrado.
3. Se define un dominio paramétrico homogéneo $u \in [0, 1]$ con `num_puntos` muestras (1500 por defecto) y se evalúan sobre él la posición y sus derivadas primera y segunda mediante `splev`.
4. Se calcula la distancia acumulada a lo largo de la curva suavizada, s , integrando la longitud de arco entre puntos consecutivos.
5. Se calcula la curvatura κ de forma analítica a partir de las derivadas paramétricas:

$$\kappa(u) = \frac{|x'(u)y''(u) - y'(u)x''(u)|}{(x'(u)^2 + y'(u)^2)^{3/2}} \quad (4.1)$$

y el radio de curvatura asociado como $R(u) = 1/\kappa(u)$ (indefinido donde $\kappa(u) = 0$, es decir, en los tramos rectos).

El resultado es un `DataFrame` (`spline_track`) con, para cada punto muestreado del trazado: el parámetro u , la distancia acumulada s , las coordenadas (X, Y) , las derivadas (dx, dy, ddx, ddy) , la curvatura, el radio de curvatura y un “Índice de Complejidad de Curva” (ICC, equivalente a la curvatura), junto con un diccionario de metadatos (`raw_points`, `spline_points`, `track_length_m`). Este artefacto permite caracterizar la dificultad geométrica del circuito (identificación de curvas cerradas, chicanes, rectas) de forma independiente de la telemetría de una vuelta concreta.

4.1.7 Condiciones meteorológicas: `condiciones_meteorologicas.py`

La extracción de condiciones ambientales sigue el mismo patrón que el resto del módulo: `extraer_condiciones` obtiene el `DataFrame` `session.weather_data` y lo enriquece con las columnas de trazabilidad `Year`, `GrandPrix` y `Session`. La limpieza, en `limpiar_condiciones`, aplica una secuencia de transformaciones algo más completa que en la versión inicial del módulo:

1. Elimina las filas completamente vacías (`dropna(how="all")`).
2. Elimina duplicados exactos, que pueden aparecer si la API devuelve lecturas repetidas.
3. Descarta las filas sin valor de temperatura de aire (`AirTemp`) o de pista (`TrackTemp`), consideradas canales críticos para el análisis, comprobando previamente su existencia para tolerar variaciones de esquema.
4. Ordena cronológicamente las lecturas por la columna `Time`, si está disponible, para garantizar la coherencia temporal del artefacto exportado.

4.1.8 Geometría y longitud del circuito: `circuito.py`

La función `estimar_longitud_circuito` calcula una aproximación de la longitud del trazado a partir de la telemetría de la vuelta más rápida entre las vueltas rápidas de la sesión (`session.laps.pick_quicklaps().pick_fastest()`), tomando el valor máximo de la columna `Distance` de su telemetría. Si la sesión no dispone de vueltas rápidas válidas, se lanza una excepción explícita, ya que el resto de cálculos geométricos dependen de esta longitud de referencia.

Se trata, explícitamente, de una *aproximación*: el valor obtenido depende de la trazada concreta seguida por el piloto en esa vuelta (que puede diferir ligeramente de la longitud oficial u homologada del circuito) y de la resolución de muestreo de la telemetría de FastF1. Esta limitación queda documentada como comentario en el propio código (*docstring* de la función) para que no se interprete como una medida oficial del trazado, sino como una referencia interna coherente para el resto del pipeline.

4.1.9 Persistencia de resultados: `almacenar_csv.py`

La función `guardar_csv(df, filename, output_dir)` centraliza el volcado de cualquier `DataFrame` a disco en formato CSV (sin índice), creando el directorio de destino si no existe. Esta pequeña utilidad

evita duplicar la lógica de exportación en cada etapa del pipeline y homogeneiza el formato de salida de todos los artefactos.

4.1.10 Artefactos generados

La ejecución completa del pipeline sobre la sesión configurada genera los ficheros recogidos en la Tabla 4.3, en `csv_data/`:

Tabla 4.3. Artefactos generados por el pipeline de extracción y limpieza.

Fichero	Contenido
<code>laps_clean.csv</code>	Tiempos de vuelta depurados (sin <i>pit laps</i> ni atípicos), con tiempos de sector, compuesto y trazabilidad.
<code>telemetry_3_mejores.csv</code>	Telemetría cruda limpia de las tres mejores vueltas de cada piloto.
<code>telemetry_piloto.csv</code>	Telemetría cruda limpia de las mejores vueltas del piloto seleccionado.
<code>weather.csv</code>	Condiciones meteorológicas de la sesión.
<code>telemetry_piloto_interpolada.csv</code>	Telemetría del piloto regularizada a paso temporal constante (100 Hz) mediante <i>splines</i> cúbicos 1D.
<code>curvatura_interpolada.csv</code>	Geometría 2D del trazado reconstruida mediante <i>spline</i> paramétrico, con curvatura y radio de curvatura por punto.
<code>circuit_info.txt</code>	Metadatos de control: año, Gran Premio, sesión y longitud estimada del circuito.
<code>arima/telemetry_todas_vueltas.csv</code>	Telemetría cruda limpia de <i>todas</i> las vueltas limpias del piloto (no solo las tres mejores), necesaria para los experimentos de generalización de <code>Modelado_arima</code> (Sección 4.5).
<code>arima/telemetry_todas_vueltas_interpolada.csv</code>	Su versión regularizada a 100 Hz.

Todos los artefactos tabulares incorporan columnas de trazabilidad (`Year`, `GrandPrix`, `Session` y, cuando procede, `Driver`, `LapID`), lo que permite combinarlos de forma inequívoca con datos de

otras sesiones en fases posteriores del proyecto.

4.1.11 Limitaciones

El pipeline se ejecuta sobre una única sesión por invocación, seleccionada mediante los argumentos de línea de comandos descritos en la Sección 4.1.3 (año, Gran Premio, tipo de sesión y piloto de referencia), por lo que el análisis de múltiples sesiones o temporadas requiere invocar el script una vez por cada una y consolidar posteriormente los CSV resultantes.

4.2 Análisis Exploratorio de Datos

Una vez depurados y estructurados los datos en la fase anterior, se lleva a cabo un análisis exploratorio (EDA) sobre los artefactos de `csv_data/`, con el objetivo de caracterizar el rendimiento de pilotos y escuderías, describir el comportamiento cinemático del monoplaza y detectar relaciones preliminares entre variables –velocidad, compuesto de neumático, condiciones meteorológicas– antes de abordar el modelado predictivo. Este bloque se corresponde con la fase “Análisis exploratorio” del anteproyecto del TFG, centrada en el estudio de la velocidad, el cálculo de estadísticas descriptivas, la generación de gráficas y mapas de calor, y las relaciones piloto–equipo–tiempo.

Al igual que en la fase de extracción y limpieza, el análisis se ha implementado de forma modular en un nuevo paquete, `Analisis_exploratorio`, orquestado por un único script `main.py` que, al no necesitar variantes de ejecución –a diferencia de `Extraccion_limpieza` o `Analisis_trazado`, que sí exponen varios modos– fija directamente sus rutas de entrada y salida sin necesidad de argumentos de línea de comandos. Carga los CSV de entrada mediante `pandas` [27], calcula los estadísticos descriptivos con `NumPy` [28] y genera las figuras y tablas resumen con `Matplotlib` [29] y `Seaborn` [30], persistiéndolas en disco. El pipeline se ha ejecutado y verificado íntegramente sobre los datos reales de la sesión de referencia (Gran Premio de Italia 2023, carrera), produciendo 15 figuras y 4 tablas sin errores.

4.2.1 Rendimiento por vuelta (`estadisticas_vueltas.py`)

A partir de `laps_clean.csv` se calculan estadísticos descriptivos (número de vueltas, media, mediana, desviación típica, mínimo y máximo de `LapTime_s`) agrupados por piloto y por equipo mediante las funciones `resumen_por_piloto` y `resumen_por_equipo`, y se generan las siguientes visualizaciones:

- **Distribución de tiempos de vuelta** por piloto y por equipo (*boxplots*, ordenados por mediana), que permiten identificar de un vistazo el ritmo relativo y la consistencia de cada piloto.
- **Evolución del ritmo de carrera:** tiempo de vuelta frente al número de vuelta para los pilotos con mejor mediana, que revela tendencias de degradación o mejora a lo largo de la carrera.
- **Efecto del compuesto de neumático:** *boxplot* del tiempo de vuelta por Compound.
- **Degradación del neumático:** diagrama de dispersión de `TyreLife` frente a `LapTime_s`, coloreado por compuesto, que aproxima visualmente la pérdida de rendimiento con el uso del neumático.
- **Tiempos por sector:** *boxplot* de `Sector1Time`, `Sector2Time` y `Sector3Time` (convertidos a segundos) para los pilotos más rápidos, que permite localizar en qué tramo del circuito se concentran las diferencias de rendimiento.

A modo de ejemplo, para la sesión de referencia el piloto con mejor tiempo de vuelta mediano es Max Verstappen (`VER`, mediana de 86.05 s sobre 49 vueltas), seguido muy de cerca por `PER`, `SAI`, `RUS` y `LEC`, todos ellos con medianas por debajo de 86.6 s.

La Figura 4.2 muestra el efecto del compuesto de neumático sobre el tiempo de vuelta en la sesión de referencia.

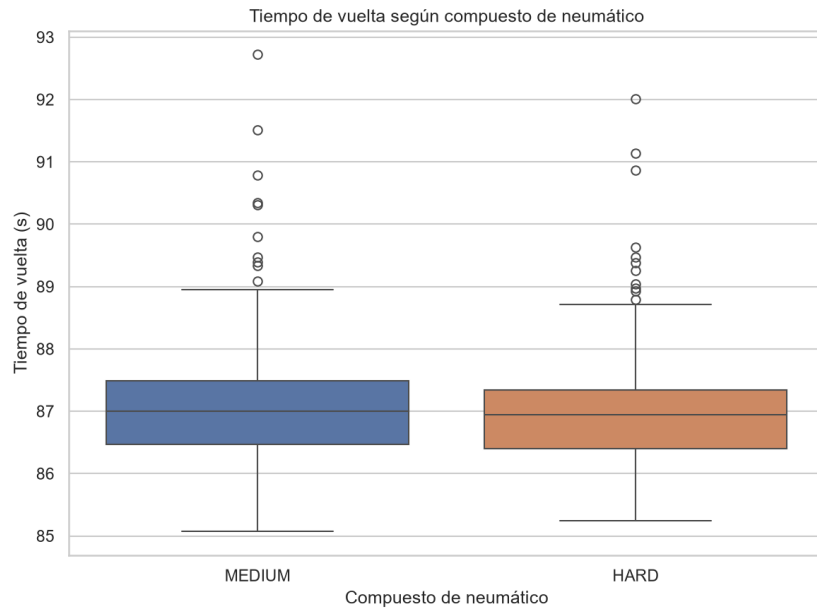


Figura 4.2. Distribución del tiempo de vuelta según el compuesto de neumático.

4.2.2 Telemetría: velocidad, acelerador, freno y marchas (estadísticas_ telemetria.py)

Sobre `telemetry_3_mejores.csv` y `telemetry_piloto.csv` se caracteriza el comportamiento cinemático del monoplaza:

- **Resumen de velocidad por piloto** (media, máxima y desviación típica), calculado con `resumen_velocidad_por_piloto`.
- **Perfil de velocidad frente a distancia:** para cada piloto se selecciona su vuelta más rápida (LapID con LapTime_s mínimo) y se traza la velocidad a lo largo del trazado, permitiendo comparar directamente el estilo de conducción y las zonas de frenada de distintos pilotos sobre el mismo circuito (Figura 4.3).
- **Distribución de velocidad:** histograma de todos los puntos de telemetría de la sesión.
- **Perfil de acelerador y freno:** evolución de `Throttle` y `Brake` a lo largo de la distancia para una vuelta de referencia, que identifica visualmente los puntos de frenada y las zonas de aceleración plena.
- **Velocidad máxima por piloto:** diagrama de barras con la velocidad punta alcanzada por cada

piloto en la sesión.

- **Uso relativo del DRS:** porcentaje de puntos de telemetría con el alerón móvil activo por piloto. En la codificación del canal DRS de FastF1, los valores ≥ 10 (10/12/14) indican el DRS realmente desplegado, mientras que valores inferiores solo indican que el sistema está disponible o deshabilitado; este umbral se aplica de forma explícita en el código (DRS_ACTIVO_UMBRAL = 10).
- **Distribución de marchas:** número de puntos de telemetría registrados en cada marcha (nGear), que refleja el perfil de velocidades típico del circuito.

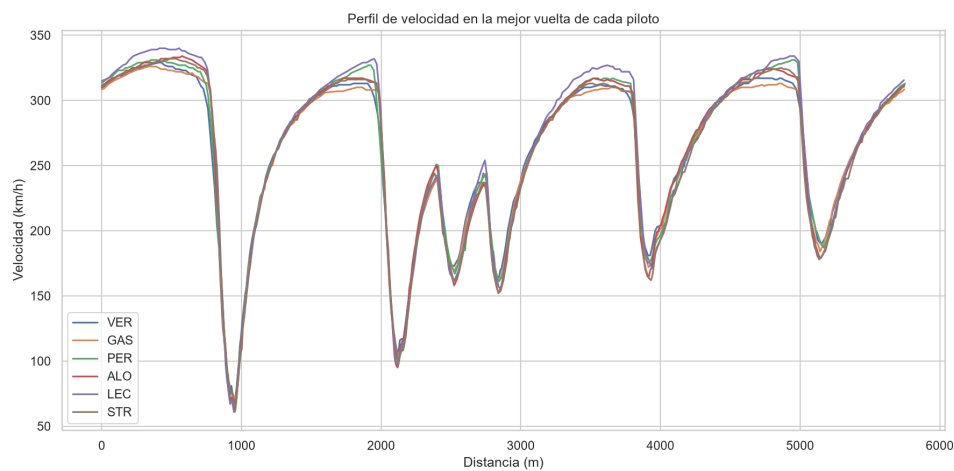


Figura 4.3. Perfil de velocidad a lo largo del trazado en la mejor vuelta de cada piloto.

4.2.3 Condiciones meteorológicas (estadisticas_meteorologicas.py)

Sobre `weather.csv` se calcula un resumen estadístico (`describe`) de temperatura del aire, temperatura de pista, humedad, velocidad del viento y presión, y se representa su evolución a lo largo de la sesión mediante un panel de series temporales.

4.2.4 Relaciones piloto-equipo-tiempo: mapas de calor (heatmaps.py)

Siguiendo uno de los objetivos explícitos del anteproyecto, se generan dos mapas de calor:

- **Ritmo de carrera piloto $\times$ vuelta** (Figura 4.4): tabla dinámica (`pivot_table`) de `LapTime_s` con los pilotos en filas y el número de vuelta en columnas, coloreada con una escala divergente

(verde = rápido, rojo = lento), que permite identificar de un vistazo tanto el ritmo relativo de cada piloto como fases específicas de la carrera (por ejemplo, vueltas bajo condiciones de seguridad o tras una parada en boxes, visibles como huecos o celdas vacías en el mapa).

- **Correlación entre variables de telemetría** (Figura 4.5): matriz de correlación de Pearson entre velocidad, acelerador, RPM y marcha, que cuantifica la relación –esperable, pero útil como comprobación de consistencia de los datos– entre estas variables físicas del monoplaza.

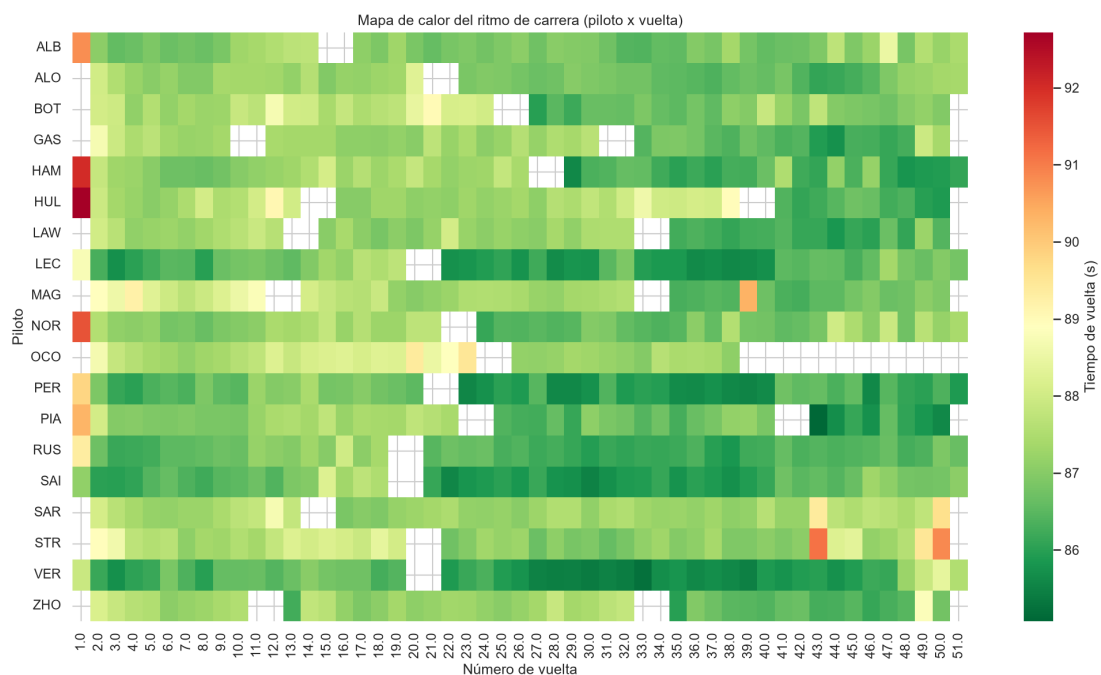


Figura 4.4. Mapa de calor del ritmo de carrera (piloto $\times$ número de vuelta).

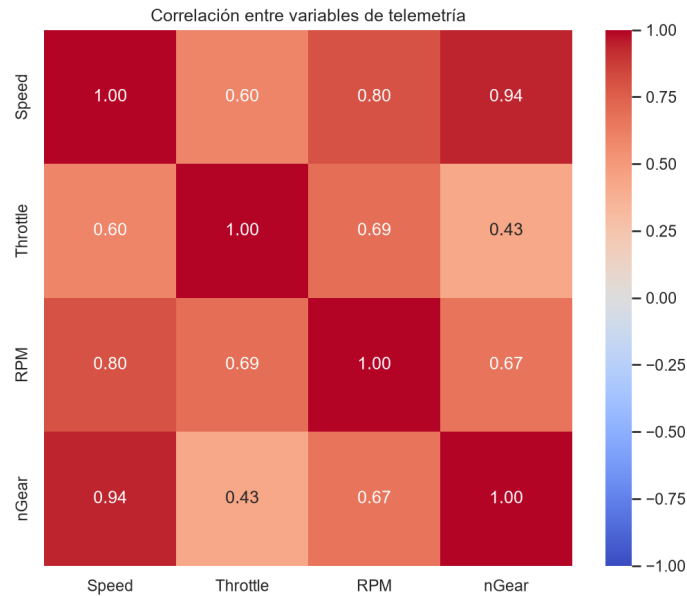


Figura 4.5. Correlación entre variables de telemetría (velocidad, acelerador, RPM y marcha).

4.2.5 Artefactos generados

La Tabla 4.4 recoge la totalidad de figuras y tablas generadas por el pipeline de análisis exploratorio sobre la sesión de referencia.

4.3 Análisis de la Dificultad del Trazado

Como fase independiente del análisis exploratorio general, y siguiendo la fase homónima del anteproyecto, se caracteriza la dificultad geométrica del trazado a partir de la reconstrucción paramétrica 2D generada durante la fase de extracción y limpieza (artefacto `curvatura_interpolada.csv`), con el objetivo de identificar y delimitar las curvas del circuito y relacionar su severidad geométrica con el rendimiento del monoplaza en fases posteriores del proyecto.

4.3.1 Corrección de escala en la geometría reconstruida

Durante la validación de este análisis se detectó una inconsistencia de unidades en `interpolar_geometria_y_telemetria_circuito`: los canales de posición X/Y que proporciona FastF1 están expresados en decímetros (1/10 m) y no en metros, mientras que el resto del *pipeline* (p. ej. `esti-`

Tabla 4.4. Artefactos generados por el pipeline de análisis exploratorio.

Módulo	Artefacto	Contenido
estadisticas_vueltas.py	resumen_por_piloto.csv / resumen_por_equipo.csv	Estadísticos descriptivos de LapTime_s por piloto y equipo.
	boxplot_tiempos_por_piloto.png / ..._por_equipo.png	Distribución de tiempos de vuelta.
	evolucion_ritmo_carrera.png	Evolución del ritmo por vuelta (top 6 pilotos).
	boxplot_tiempo_por_compuesto.png	Tiempo de vuelta por compuesto.
	scatter_degradacion_neumatico.png	Tiempo de vuelta vs. vida del neumático.
	boxplot_tiempos_sector.png	Tiempos por sector (top 8 pilotos).
	resumen_velocidad_por_piloto.csv	Media, máxima y desviación de velocidad por piloto.
	perfil_velocidad_distancia.png	Perfil de velocidad en la mejor vuelta de cada piloto.
estadisticas_telemetria.py	histograma_velocidad.png	Distribución de velocidad.
	perfil_acelerador_freno.png	Perfil de acelerador/freno vs. distancia.
	barplot_velocidad_maxima.png	Velocidad máxima por piloto.
	barplot_uso_drs.png / distribucion_marchas.png	Uso relativo de DRS y distribución de marchas.
	resumen_meteorologico.csv	Estadísticos descriptivos de las variables meteorológicas.
estadisticas_meteorologicas.py	evolucion_meteorologia.png	Evolución temporal de temperatura, humedad y viento.
	heatmap_ritmo_piloto	Ritmo de carrera piloto × vuelta.

`mar_longitud_circuito`, basada en el canal `Distance`) trabaja en metros. Al no reescalar las coordenadas antes de ajustar el *spline* paramétrico, la longitud de arco s , la curvatura κ y el radio de curvatura resultantes quedaban distorsionados por un factor de $10\times$: la longitud reconstruida del trazado llegaba a 57613 m, frente a los 5758 m reales estimados de forma independiente a partir del canal `Distance`. Un circuito de 5.8 km de longitud no puede físicamente ocupar una caja delimitadora de $12.5\text{ km} \times 21.7\text{ km}$, lo que hacía evidente el error de escala.

La corrección consiste en dividir las coordenadas X/Y entre 10 antes de ajustar el *spline* (`eje_x = df_tel["X"].values / 10.0`), de forma que todas las magnitudes derivadas (longitud de arco, curvatura, radio) queden expresadas de forma coherente en metros. Tras la corrección, la longitud reconstruida del trazado es de 5759.16 m, consistente con la estimación independiente de 5758.34 m, y el radio mínimo de curvatura observado (14.2 m) es geoméricamente plausible para una chicane real.

4.3.2 Detección de curvas (`Analisis_trazado/deteccion_curvas.py`)

Sobre la geometría corregida, la función `detectar_curvas` segmenta el trazado en tramos de curva contiguos aplicando un umbral sobre el radio de curvatura local: un punto se considera parte de una curva si $\text{Radius_m} < 300\text{ m}$, valor empírico que, para el trazado de referencia, produce una segmentación estable en un rango amplio de umbrales (entre 150 y 500 m se obtienen sistemáticamente entre 10 y 11 tramos). Dado que la geometría reconstruida es un bucle cerrado, el algoritmo rota internamente el origen del recorrido a un punto de recta (si existe) antes de agrupar los tramos contiguos, evitando partir en dos una curva que cruce el cierre del bucle ($s = 0 / s = s_{\text{max}}$).

Para cada curva detectada se registra el radio mínimo (ápex) y su posición (X, Y, distancia acumulada s). La función `clasificar_dificultad` asigna a cada curva una categoría relativa –Baja, Media o Alta– según los terciles del radio mínimo de todas las curvas detectadas en la sesión. Se trata, por tanto, de una escala *relativa al propio trazado*: no implica una comparación absoluta con la dificultad de curvas de otros circuitos.

Sobre el trazado de referencia (GP de Italia 2023) se detectan 11 curvas, que se corresponden de forma directa con el trazado real del Autódromo Nazionale di Monza (Tabla 4.5): la variante del Rettifilo, la Curva Grande, la variante della Roggia, Lesmo 1 y 2, la variante Ascari (una chicane triple) y la Parabólica. Esta correspondencia –once curvas detectadas de forma puramente geométrica, coincidentes en número y posición relativa con las once curvas nombradas oficialmente del circuito–

constituye una validación indirecta tanto de la corrección de escala aplicada como del propio método de segmentación.

Tabla 4.5. *Curvas detectadas en el trazado de referencia (GP de Italia 2023).*

Nº	Nombre	Radio mín. (m)	Distancia ápex (m)	Dificultad
1	Rettifilo	24.3	898.4	Alta
2	Rettifilo	14.2	932.7	Alta
3	Curva Grande	296.6	1328.2	Baja
4	Roggia	34.0	2108.1	Alta
5	Roggia	46.8	2131.5	Alta
6	Lesmo 1	62.3	2488.0	Media
7	Lesmo 2	56.8	2808.3	Media
8	Ascari	49.1	3883.4	Media
9	Ascari	97.1	4022.1	Baja
10	Ascari	79.1	4099.1	Baja
11	Parabólica	71.0	5132.7	Baja

4.3.3 Mapa de dificultad del trazado (`Analisis_trazado/mapa_trazado.py`)

A partir de la segmentación anterior se generan dos visualizaciones:

- **Mapa de dificultad continuo** (Figura 4.6): el trazado completo se dibuja como una colección de segmentos de línea (`matplotlib.collections.LineCollection`) coloreados de forma continua según el radio de curvatura local, acotado a un máximo de 600 m para dar contraste visual entre rectas y curvas. Tras varias iteraciones se ajustó la paleta de color: primero se sustituyó la paleta divergente inicial (rojo-amarillo-verde, `RdYlGn_r`) –poco legible, ya que el rojo y el verde tienen una luminosidad similar y la dificultad apenas se apreciaba– por una paleta secuencial de un solo tono (`YlOrRd`); al comprobarse que su extremo amarillo más claro se confundía con el fondo blanco de la figura, se optó finalmente por una paleta secuencial rosa-morado (`RdPu`, sin ningún tono amarillo), recortada en su 25 % más claro, de forma que incluso las rectas (dificultad mínima) quedan en un rosa claro pero visible, y las curvas más cerradas (dificultad máxima) en morado oscuro.

- **Mapa de curvas numeradas** (Figura 4.7): se superponen marcadores en el ápex de cada curva detectada, coloreados según su categoría de dificultad relativa, junto con el nombre real de cada curva (específico del circuito de Monza) conectado mediante una línea guía desplazada hacia fuera del trazado, para evitar solapar curvas muy próximas entre sí como ocurre en las chicanes.

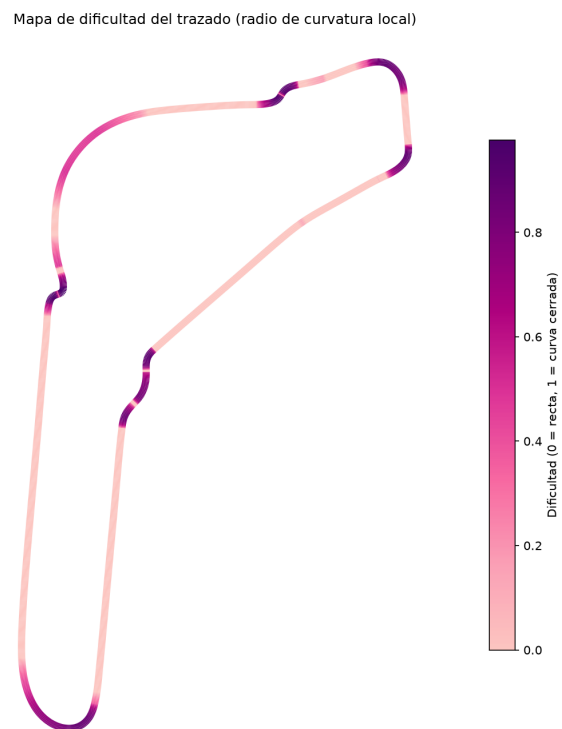


Figura 4.6. Mapa de dificultad continuo del trazado, coloreado según el radio de curvatura local.

4.3.4 Relación entre dificultad geométrica y comportamiento real (Análisis_trazado/perfil_velocidad_curvas.py)

Para ello, `detectar_curvas` se amplía para registrar, además del ápex, el rango de distancia de cada tramo de curva (s_{inicio} , s_{fin}). A continuación, para cada curva y cada una de las vueltas disponibles del piloto en `telemetry_piloto_interpolada.csv` (identificadas por LapID), se

filtran los puntos de telemetría cuyo **Distance** cae dentro de ese rango y se calculan:

- **Velocidad mínima real:** el mínimo de **Speed** dentro del tramo, es decir, la velocidad de paso real por el ápex.
- **Velocidad de entrada:** el máximo de **Speed** en una ventana de 100 m previa al inicio del tramo, que aproxima la velocidad alcanzada antes de iniciar la frenada. Esta ventana se calcula de forma modular sobre la distancia acumulada para tratar correctamente el caso de curvas situadas justo después de la línea de meta, donde la ventana previa cruza el cierre de vuelta.
- **Caída de velocidad en frenada:** la diferencia entre la velocidad de entrada y la velocidad mínima real.
- **Uso de freno:** el porcentaje de puntos del tramo con el canal **Brake** activo (> 0.5).

Estas cuatro métricas se promedian entre las vueltas disponibles del piloto (tres, en la configuración de referencia) y se incorporan como columnas adicionales a `resumen_curvas_trazado.csv` (Tabla 4.6).

Tabla 4.6. Perfil de velocidad real por curva (GP de Italia 2023, HAM), cruzando la segmentación geométrica con la telemetría interpolada.

Nº	Nombre	Radio (m)	Dificultad	Vel. mín. (km/h)	Frenada Δ (km/h)	Freno (%)	<i>r</i>
1	Rettifilo	24.3	Alta	75.8	238.9	88.5	0.804
2	Rettifilo	14.2	Alta	61.9	161.3	10.4	
3	Curva Grande	296.6	Baja	278.1	-0.1	0.0	
4	Roggia	34.0	Alta	103.4	209.7	87.6	
5	Roggia	46.8	Alta	102.6	138.6	0.0	
6	Lesmo 1	62.3	Media	170.4	75.4	28.7	
7	Lesmo 2	56.8	Media	159.6	80.4	29.2	
8	Ascari	49.1	Media	171.5	130.4	55.9	
9	Ascari	97.1	Baja	175.0	64.2	0.0	
10	Ascari	79.1	Baja	226.5	-0.2	0.0	
11	Parabólica	71.0	Baja	185.9	125.5	18.5	

La última columna recoge el coeficiente de correlación de Pearson entre **Radio_min_m** y **Velocidad_min_kmh** calculado sobre las 11 curvas, persistido en `resumen_curvas_trazado.csv` como

la columna R_Radio_Velocidad (idéntico en todas las filas, al tratarse de un estadístico global de la tabla y no de una métrica por curva).

La Figura 4.8 representa el radio mínimo de curvatura frente a la velocidad mínima real por curva, coloreada según la categoría de dificultad geométrica. Se observa una relación positiva clara entre ambas variables, con un coeficiente de correlación de Pearson $r = 0.804$: las curvas clasificadas como de dificultad Alta (radios más cerrados) presentan sistemáticamente las velocidades reales más bajas, y las de dificultad Baja las más altas. Este resultado valida empíricamente que la clasificación de dificultad basada únicamente en geometría —calculada sin ningún dato de telemetría— se corresponde con el comportamiento real del monoplaza en pista, y aporta evidencia cuantitativa directa sobre la relación entre velocidad y dificultad del trazado planteada como objetivo en el anteproyecto de este TFG.

Las curvas 3 y 10 (Curva Grande y la última chicane de Ascari) constituyen un caso ilustrativo: su radio relativamente amplio se traduce en un Δ de frenada prácticamente nulo y un uso de freno del 0 %, es decir, se toman a fondo sin frenar, coherente con su clasificación de dificultad Baja. En el extremo opuesto, la variante del Rettifilo (curvas 1 y 2) combina el radio más cerrado del trazado con el mayor uso de freno registrado (88.5 % y una caída de velocidad de 238.9 km/h), consistente con tratarse de la frenada más exigente de todo el circuito de Monza tras la recta principal.

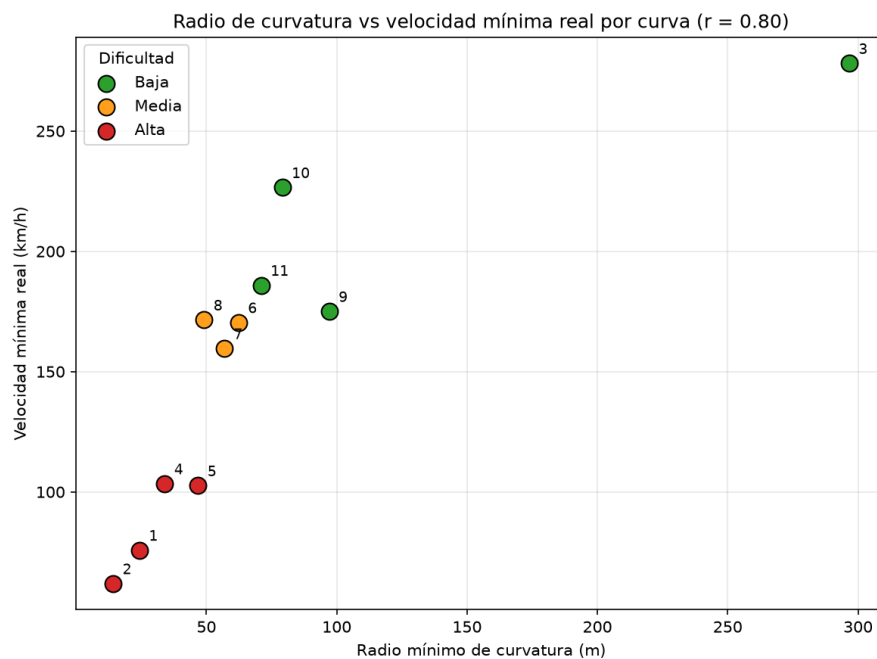


Figura 4.8. Radio mínimo de curvatura frente a velocidad mínima real por curva ($r = 0.804$), coloreada por dificultad geométrica.

Esta validación se apoya únicamente en las tres mejores vueltas del piloto de referencia, por lo que las métricas de velocidad de entrada y uso de freno están sujetas a la variabilidad propia de una muestra pequeña.

4.3.5 Validación multi-circuito de la reconstrucción geométrica

Todo el análisis anterior se ha desarrollado y ajustado sobre un único circuito (Monza), por lo que se valida aquí si la reconstrucción geométrica por *spline* paramétrico (`interpolar_geometria_y_telemetria_circuito`, Sección 4.1.3) generaliza a otros trazados sin reajustar sus parámetros (suavizado = 80, umbral de radio = 300 m, ambos calibrados únicamente para Monza). Siguiendo el diseño extracción/análisis del resto del proyecto, la validación se divide en dos módulos independientes que se comunican por CSV, sin que ninguno importe código del otro: `Extraccion_limpieza/main.py --modo multicircuito` descarga la sesión de cada circuito, reconstruye su geometría y persiste `curvatura_interpolada.csv` en un subdirectorío propio por circuito (`csv_data/multicircuito/<circuito>/`); y `Analisis_trazado/main.py --modo multicircuito` lee esos artefactos y aplica la misma detección de curvas y visualización ya empleadas para Monza. Ambos scripts mantienen internamente la lógica de extracción y de validación en ficheros propios (`extraer_multicircuito.py` y `validacion_multicircuito.py`, respectivamente), pero cada carpeta expone un único punto de entrada ejecutable (`main.py`), coherente con el resto de módulos del proyecto.

Se seleccionaron cuatro circuitos de la temporada 2023 con una hipótesis de partida: dos donde cabía esperar que la reconstrucción funcionase bien, por tener curvas amplias y bien separadas entre sí (Spa-Francorchamps, Silverstone), y dos donde cabía esperar que se degradase, por tratarse de circuitos urbanos con curvas muy cerradas y próximas entre sí (Mónaco, Singapur). Como referencia de la longitud real de cada trazado se emplea, igual que en la Sección 4.3, la estimación independiente basada en el canal `Distance` (`estimar_longitud_circuito`).

La Tabla 4.7 resume los resultados de esta validación, que solo confirman parcialmente la hipótesis de partida. En error de longitud, los cuatro circuitos reconstruyen su trazado con una precisión razonable (siempre por debajo del 0.6 %), incluido Mónaco (0.10 %, incluso mejor que Silverstone o Singapur). Sin embargo, el número de curvas detectadas en Mónaco (27) es muy superior a las 19 curvas oficiales del circuito, evidenciando una clara sobre-segmentación: el umbral fijo de 300 m, calibrado para las proporciones de Monza, fragmenta en varios tramos discontinuos curvas que de-

Tabla 4.7. Validación de la reconstrucción geométrica en cuatro circuitos de la temporada 2023.

Circuito	Hipótesis	L. indep. (m)	L. spline (m)	Error (%)	Curvas reales	Curvas detect.
Spa-Francorchamps	bien	6947.9	6949.1	0.02	19	19
Silverstone	bien	5801.4	5828.2	0.46	18	18
Mónaco	mal	3274.9	3271.5	0.10	19	27
Singapur	mal	4889.1	4862.8	0.54	19	19

berían contarse como una sola cuando su radio fluctúa por encima y por debajo del umbral dentro de una misma curva real –algo mucho más probable en un trazado a la escala de Mónaco, con radios mínimos de hasta 6 m (la horquilla de Fairmont, la curva más cerrada del calendario), que en circuitos de mayor escala–. Spa y Silverstone, en cambio, reproducen con notable fidelidad su número real de curvas (19 y 18, respectivamente, coincidiendo con las cifras oficiales de ambos trazados). Contrariamente a la hipótesis inicial, Singapur –pese a ser también un circuito urbano de curvas cerradas– reproduce igualmente sus 19 curvas oficiales sin evidencia de sobre-segmentación, probablemente porque, a diferencia de Mónaco, sus curvas mantienen radios mínimos algo mayores (Figura 4.10b) y están más regularmente espaciadas.

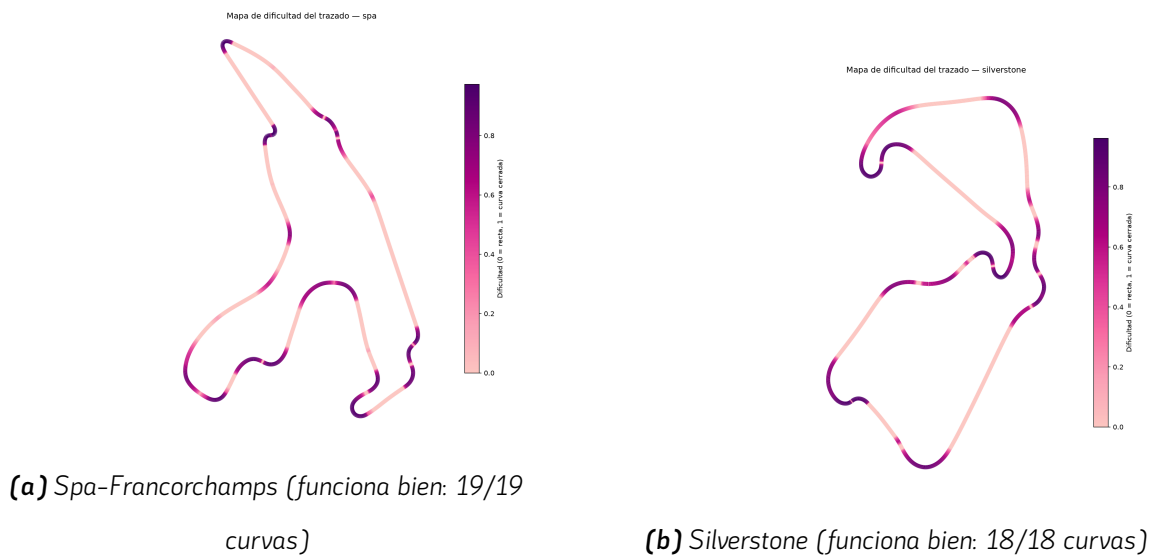
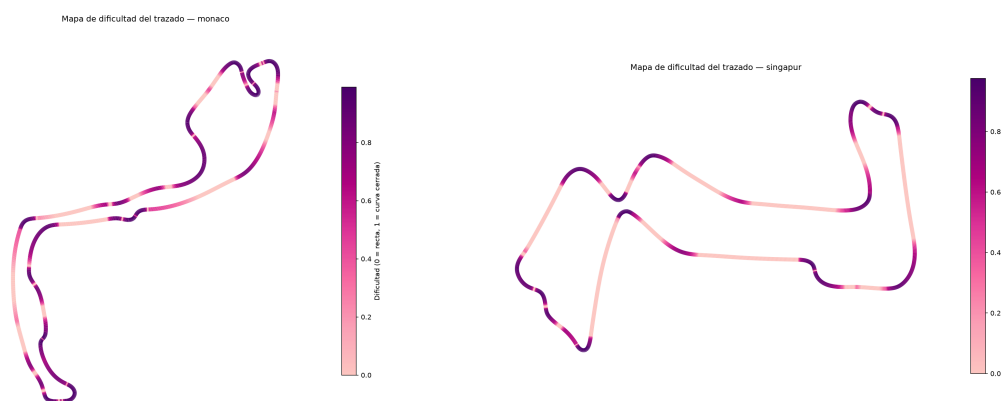


Figura 4.9. Reconstrucción geométrica en los dos circuitos donde el método generaliza correctamente.



(a) Mónaco (sobre-segmenta: 27 curvas detectadas frente a 19 reales)

(b) Singapur (funciona bien pese a la hipótesis inicial: 19/19 curvas)

Figura 4.10. Reconstrucción geométrica en Mónaco (caso problemático) y Singapur (excepción positiva a la hipótesis inicial).

Las Figuras 4.9 y 4.10 muestran la reconstrucción geométrica resultante en los cuatro circuitos, contrastando los dos casos donde el método generaliza correctamente con los dos casos de la hipótesis inicial (uno confirmado, Mónaco; el otro refutado, Singapur). En conjunto, esta validación indica que la reconstrucción por *spline* generaliza razonablemente bien a circuitos de proporciones similares o mayores que Monza, y que su principal limitación —la sobre-segmentación por umbral fijo— aparece específicamente en circuitos con curvas extremadamente cerradas y próximas entre sí, como Mónaco, más que en circuitos urbanos en general, ya que el umbral de radio y el suavizado no se adaptan de forma proporcional a la escala de cada circuito.

4.3.6 Artefactos generados y limitaciones

El pipeline (orquestrado por `Análisis_trazado/main.py --modo trazado`, con el umbral de radio parametrizable por CLI mediante `--radio-umbral`; las rutas de entrada y salida quedan fijas, como en el resto de módulos que no necesitan variarlas entre ejecuciones) genera los artefactos recogidos en la Tabla 4.8.

El umbral de radio (300 m) y la clasificación por terciles son elecciones metodológicas razonables pero no las únicas posibles: la segmentación no se ha validado frente a los nombres oficiales de curva publicados por el circuito, y el diccionario de nombres de curva empleado es actualmente específico de Monza, sin generalizar a otros circuitos mediante una fuente de datos externa (p. ej. OpenStreetMap

Tabla 4.8. Artefactos generados por el análisis de dificultad del trazado.

Artefacto	Contenido
resumen_curvas_trazado.csv	Curva, nombre, radio mínimo, distancia del ápex, dificultad (Tabla 4.5) y, cruzando con telemetría real, velocidad mínima, velocidad de entrada, caída de velocidad, uso de freno y coeficiente r radio-velocidad (Tabla 4.6).
mapa_trazado_dificultad.png	Mapa de dificultad continuo del trazado.
mapa_trazado_curvas_numeradas.png	Curvas numeradas, coloreadas y nombradas.
relacion_radio_velocidad.png	Dispersión radio de curvatura vs. velocidad mínima real, con el coeficiente de correlación (Figura 4.8).

o los mapas oficiales de la FIA). El cruce del radio de curvatura con la velocidad y el uso de freno reales, planteado inicialmente como línea futura, se aborda ya en la Sección 4.3.4.

4.4 Análisis de Variabilidad de la Velocidad

4.4.1 Motivación y adaptación al contexto de F1

Las secciones anteriores caracterizan la variabilidad de la velocidad de forma indirecta: mediante estadísticos agregados por piloto (Sección 4.2) o mediante su relación con la geometría del trazado (Sección 4.3). Se profundiza aquí en esta variabilidad con un conjunto de métricas más ricas, tomadas de la literatura sobre variabilidad glucémica —el análisis de las fluctuaciones de la glucemia medida de forma continua (*Continuous Glucose Monitoring*, CGM) en pacientes diabéticos—, un campo que ha desarrollado, durante décadas, un catálogo de métricas específicamente diseñado para cuantificar cuánto y cómo fluctúa una serie temporal fisiológica, más allá de su media y desviación típica. La adaptación sigue el trabajo de Colás et al. [24], que compara sistemáticamente estas métricas (convencionales, de secuencialidad, de fenotipo y de análisis de fluctuación sin tendencia) para predecir riesgo de diabetes

tipo 2, y toma como referencia de implementación el repositorio `cgm-variability-analysis` [25].

La adaptación requiere, en primer lugar, decidir qué señal de F1 desempeña el papel de la “glucemia a lo largo del día” de cada paciente. Se elige el **perfil de velocidad dentro de una vuelta**, es decir, la serie $\text{Speed}(t)$ regularizada a 100 Hz ya generada en la fase de extracción y limpieza (`csv_data/arima/telemetry_todas_vueltas_interpolada.csv`, Sección 4.5.2): cada **vuelta** pasa así a jugar el papel de **paciente**, con su propia serie temporal completa, lo que permite comparar la variabilidad de la conducción entre vueltas –por compuesto de neumático, por vida del neumático o por ritmo– del mismo modo que el trabajo de referencia compara pacientes entre sí.

En segundo lugar, dos de las familias de métricas del paper –las métricas de fenotipo prediabético (TU100, tiempo bajo 100 mg/dl, y AO140, área sobre 140 mg/dl)– emplean umbrales clínicos fijos, iguales para todos los pacientes, sin equivalente físico universal en velocidad de F1. Se descartaron dos alternativas antes de fijar el criterio finalmente adoptado: (i) usar umbrales físicos fijos (p.ej. una velocidad de referencia en curva lenta y otra en recta), poco comparables entre circuitos distintos; y (ii) usar percentiles calculados sobre la propia vuelta, que resultarían tautológicamente iguales (25 %/ 75 % del tiempo, por construcción) en todas las vueltas y no discriminarían nada entre ellas. Se adopta en su lugar un criterio intermedio: los percentiles 25 y 75 de la velocidad *agregada de toda la sesión* (todas las vueltas limpias del piloto), fijos para todas las vueltas comparadas –igual que 100/140 mg/dl son fijos para todos los pacientes–, pero propios de cada sesión y no de una escala fisiológica ajena al automovilismo.

4.4.2 Arquitectura del módulo

Siguiendo el mismo principio de diseño que el resto del proyecto (Sección 4.1.3), el análisis se implementa en un nuevo paquete independiente, `Analisis_variabilidad`, que lee directamente los artefactos ya persistidos por `Extraccion_limpieza` sin importar código de ningún otro módulo:

- `metricas_variabilidad.py` – funciones puras de NumPy [28] que implementan cada métrica sobre un `array` de velocidad de una vuelta.
- `perfil_variabilidad.py` – carga de la telemetría por vuelta, cálculo de los umbrales de fenotipo agregados y aplicación de las métricas a cada vuelta (`groupby("LapID")`), junto con el cruce con `laps_clean.csv` (compuesto, vida del neumático, tiempo de vuelta).
- `graficas_variabilidad.py` – generación de las figuras.

- `main.py` – único punto de entrada ejecutable del módulo.

Un aspecto de diseño transversal a varias métricas es la **resolución temporal** a la que se calculan. Las métricas basadas en la transición entre muestras consecutivas (CONGA, GFI, Poincaré) solo tienen sentido físico si el paso temporal entre muestras corresponde a una escala real de conducción; calculadas sobre la serie bruta a 100 Hz, capturarían únicamente el micro-suavizado del *spline* de interpolación (Sección 4.1.6.3), no la dinámica de frenada o aceleración. Por ello se calculan sobre la misma serie submuestreada a 5 Hz que ya emplea `Modelado_arima` (Sección 4.5.2), reutilizando el mismo criterio (y el mismo factor, 20) por la misma razón de fondo. Las métricas convencionales, MAGE, DFA y fenotipo, en cambio, sí se calculan sobre la serie completa a 100 Hz: MAGE por estar explícitamente filtrada por un umbral de 1 SD (Sección 4.4.4), que descarta por construcción el ruido de muy alta frecuencia, y DFA por ser un método multiescala que se beneficia de disponer de más puntos y más escalas disponibles.

4.4.3 Métricas convencionales

Sobre cada vuelta se calculan la media, la desviación típica y el coeficiente de variación de la velocidad, $CV = SD / \text{media} \times 100$, expresado en porcentaje.

4.4.4 Métricas de secuencialidad: MAGE, CONGA e IFV

MAGE.

El *Mean Amplitude of Glycemic Excursions* [31] mide la amplitud media de las oscilaciones “grandes” de la serie. Se implementa el algoritmo original de puntos de giro, en lugar de la aproximación simplificada del repositorio de referencia (diferencia entre muestras consecutivas por encima de un umbral), porque esta última resulta muy sensible al ruido de alta frecuencia y subestima sistemáticamente la amplitud real de una frenada larga fragmentada en muchos pasos pequeños. El procedimiento es:

1. Se localizan los puntos de giro de la serie (máximos y mínimos locales, allí donde cambia el sentido de la pendiente).
2. Se calcula la amplitud entre cada par de puntos de giro consecutivos.

3. Se conservan únicamente las amplitudes que superan k veces la desviación típica de toda la vuelta ($k = 1$, como en el trabajo de referencia); estas "excursiones" se corresponden, en la práctica, con frenadas y curvas completas, no con oscilaciones menores.
4. MAGE es la media de las amplitudes conservadas.

CONGA.

El *Continuous Overlapping Net Glycemic Action* es la desviación típica de las diferencias entre puntos separados un retardo fijo, $CONGA = SD(x_{i+\text{lag}} - x_i)$. En el trabajo original, CONGA-2 usa un retardo de 2 horas sobre glucemia muestreada cada 5-15 minutos; aquí se aplica sobre la serie a 5 Hz con un retardo de 20 muestras (≈ 4 s), una escala "media" dentro de una vuelta, del orden de una zona de frenada o una curva completa.

IFV (adaptación de GFI).

El *Glycaemic Fluctuation Index* se define en el paper de referencia como el área entre el perfil y la media, pero se implementa en el repositorio de referencia como la media del valor absoluto de las diferencias sucesivas; se sigue aquí esta segunda definición operativa, y se renombra Índice de Fluctuación de Velocidad (IFV) para dejar constancia explícita de que no es la definición literal del paper: $IFV = \text{media}(|x_{i+1} - x_i|)$.

4.4.5 Fenotipo: TB_{p25} y AS_{p75}

Adaptación de TU100 (tiempo bajo 100 mg/dl) y AO140 (área sobre 140 mg/dl), con los umbrales de sesión descritos anteriormente (u_{bajo} , percentil 25; u_{alto} , percentil 75, ambos sobre la velocidad agregada de toda la sesión):

$$TB_{p25} = \frac{\#\{i : x_i < u_{\text{bajo}}\}}{N} \times 100 \quad (\% \text{ del tiempo de vuelta en zona lenta}) \quad (4.2)$$

$$AS_{p75} = \Delta t \sum_i \frac{\max(0, x_i - u_{\text{alto}})}{3.6} \quad (\text{metros de exceso acumulado sobre el umbral alto}) \quad (4.3)$$

AS_{p75} sigue el mismo principio que AO140 –convertir un umbral binario en una variable continua que también capture *cuánto* se supera el umbral, no solo *si* se supera–, expresado aquí en metros de velocidad excedente en lugar de mg/dl · min.

4.4.6 Diagrama de Poincaré

El diagrama de Poincaré representa cada punto de la serie frente al siguiente, (x_i, x_{i+1}) , y ajusta una elipse a la nube de puntos resultante [32]: SD1 mide la dispersión perpendicular a la diagonal de identidad (variabilidad de corto plazo, entre muestras consecutivas) y SD2 la dispersión a lo largo de ella (variabilidad de largo plazo):

$$SD1 = \frac{SD(x_{i+1} - x_i)}{\sqrt{2}}, \quad SD2 = \frac{SD(x_{i+1} + x_i)}{\sqrt{2}}, \quad E = SD2/SD1 \quad (4.4)$$

La Figura 4.11 muestra el diagrama de Poincaré de la vuelta más rápida de la sesión de referencia: la nube de puntos se extiende de forma muy alargada a lo largo de la diagonal (SD2 grande) y apenas se dispersa perpendicularmente a ella (SD1 pequeño), reflejo directo de que la velocidad entre dos muestras consecutivas a 5 Hz apenas cambia salvo en las zonas de frenada más bruscas, donde la nube se separa visiblemente de la diagonal.

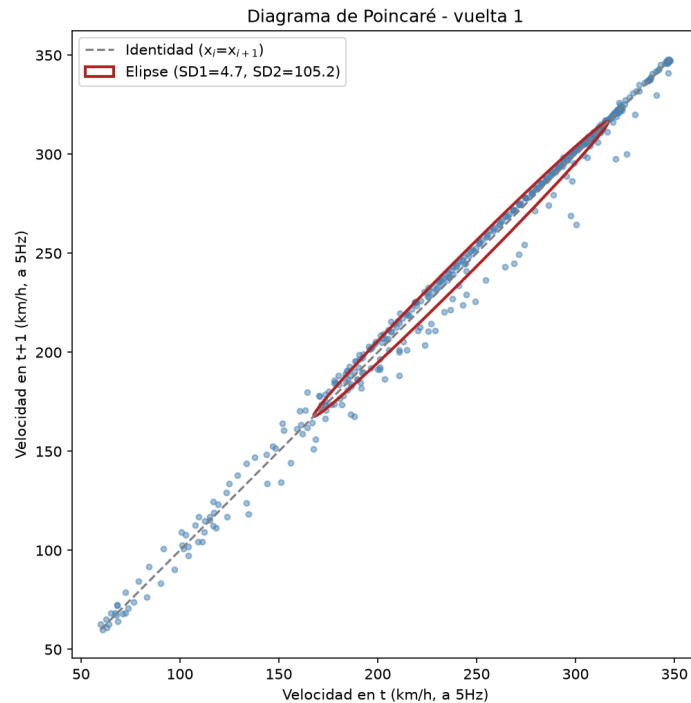


Figura 4.11. Diagrama de Poincaré de la vuelta más rápida de la sesión de referencia, con la elipse de semiejes SD1/SD2.

4.4.7 DFAint y DFAraw

El análisis de fluctuación sin tendencia (*Detrended Fluctuation Analysis*, DFA) [33] mide cómo escala la fluctuación de una serie con el tamaño de ventana de observación, mediante un exponente α : la serie se divide en ventanas de tamaño n , se ajusta y resta una tendencia lineal a cada ventana, y se calcula la fluctuación residual $F(n)$; α es la pendiente de $\log F(n)$ frente a $\log n$ en un rango de escalas logarítmicamente espaciado (4 a $N/4$ muestras, en 12 pasos). Siguiendo la distinción del trabajo de referencia, se calculan dos variantes:

- **DFAint**, la variante clásica: antes del análisis de ventanas, la serie se integra (suma acumulada de sus desviaciones respecto a la media), lo que la convierte en un perfil tipo “camino aleatorio” antes de medir su escalado.
- **DFAraw**: se omite el paso de integración y el mismo análisis de ventanas se aplica directamente sobre la serie centrada.

Un valor de $\alpha \approx 0.5$ es propio de ruido blanco (sin correlación entre muestras); $\alpha \approx 1.5$, de un camino aleatorio (altamente correlacionado, no estacionario); valores intermedios indican distintos grados de correlación de largo plazo. Esta interpretación se verificó sobre señales sintéticas antes de aplicarla a los datos reales: ruido blanco simulado produjo DFAint ≈ 0.49 , y una señal periódica pura, DFAint ≈ 1.25 , ambos coherentes con la teoría.

4.4.8 Resultados sobre la sesión de referencia

Aplicado a las 48 vueltas limpias del piloto de referencia (Sección 4.5.2), con umbrales de fenotipo $u_{\text{bajo}} = 184.4$ km/h y $u_{\text{alto}} = 307.8$ km/h (percentiles 25/75 de la velocidad agregada de la sesión), la Figura 4.12 muestra la distribución de seis de las métricas calculadas: todas presentan variación real entre vueltas (por ejemplo, el CV de velocidad oscila entre 29.5 % y 33.8 %, y MAGE entre 127.7 y 176.1 km/h), confirmando que no son degeneradas ni constantes en la práctica.

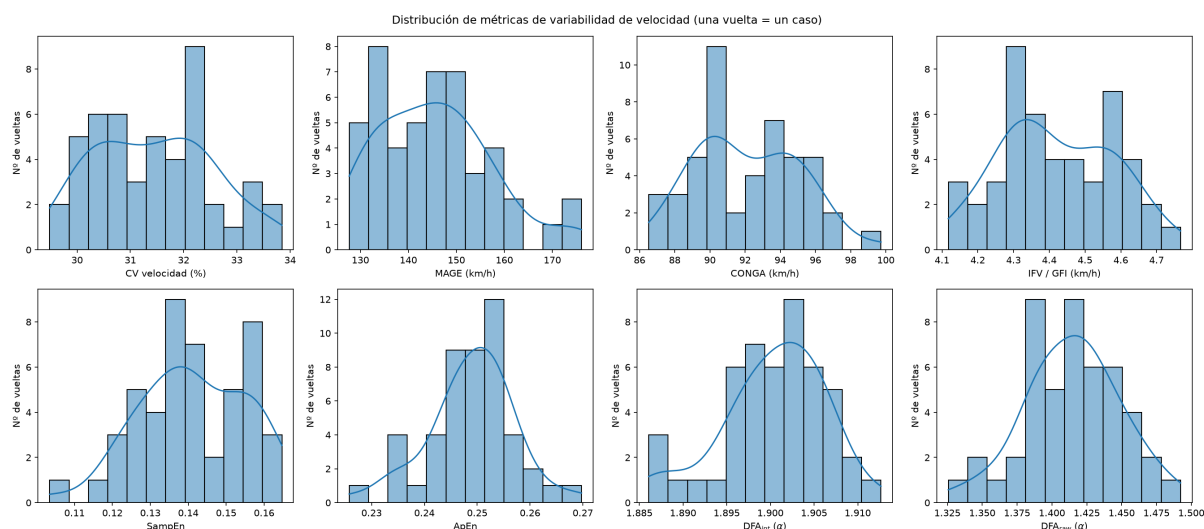


Figura 4.12. Distribución, entre las 48 vueltas de la sesión de referencia, de seis métricas de variabilidad de velocidad.

Para valorar si estas métricas capturan información relevante más allá de la variabilidad ya conocida, se calcula su correlación de Pearson con el tiempo de vuelta (`LapTime_s`) y con la vida del neumático (`TyreLife`) de cada vuelta (Tabla 4.9).

Tabla 4.9. Correlación de Pearson de cuatro métricas de variabilidad (una por familia) con el tiempo de vuelta y la vida del neumático.

Métrica	Familia	$r(\text{LapTime_s})$	$r(\text{TyreLife})$
CV_Speed_pct	Convencional	0.670	0.328
MAGE	Secuencialidad	0.370	−0.058
DFAint_alpha	DFA	0.043	−0.521
SD1	Poincaré	0.054	0.498

Los resultados sugieren dos hallazgos con sentido físico. Primero, el CV de velocidad ($r = 0.670$) correlaciona de forma notable con el tiempo de vuelta: las vueltas más lentas son también las más irregulares —más dispersas relativamente— en su perfil de velocidad, coherente con vueltas afectadas por tráfico o errores puntuales de pilotaje, en las que la velocidad dentro de la vuelta se aparta más de un patrón repetitivo. Segundo, DFAint ($r = -0.521$) y SD1 ($r = 0.498$) correlacionan, en cambio, con la vida del neumático y apenas con el tiempo de vuelta: a medida que el neumático se degrada, el perfil de velocidad se vuelve menos correlacionado a largo plazo (DFAint decrece) y algo más

ruidoso muestra a muestra (SD1 crece), un patrón distinto del que captura la métrica convencional, y que apunta a que DFA y Poincaré aportan información complementaria –no redundante– sobre cómo degrada el neumático el estilo de conducción, más allá de su efecto ya conocido sobre el tiempo de vuelta en sí (Sección 4.2). La Figura 4.13 ilustra esta segunda relación.

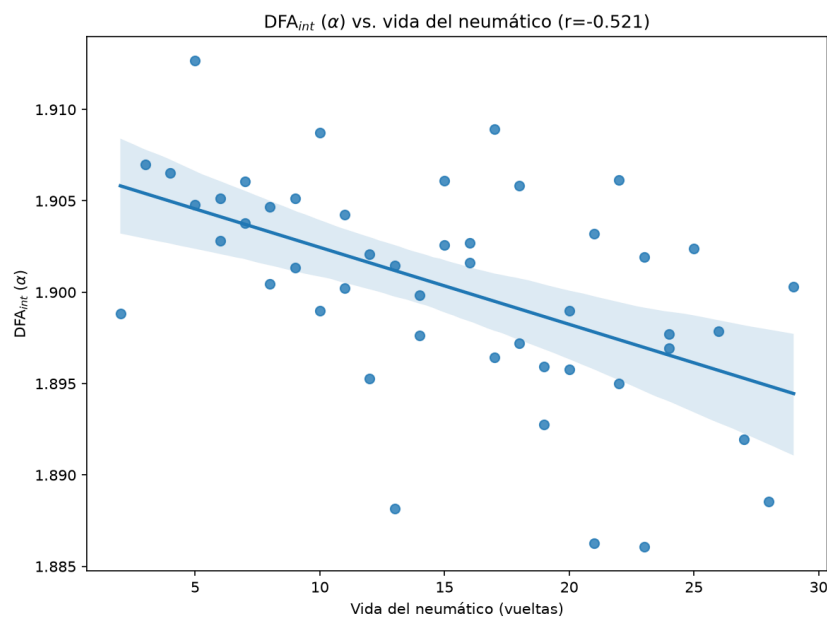


Figura 4.13. DFA_{int} frente a la vida del neumático ($r = -0.521$).

Por último, siguiendo el planteamiento de comparación de grupos del trabajo de referencia (que compara pacientes con y sin diabetes tipo 2), se compara la distribución de cada métrica entre las dos vueltas de compuesto de neumático disponibles en la sesión (25 vueltas con neumático duro, 23 con medio). La Figura 4.14 muestra esta comparación para el CV de velocidad.

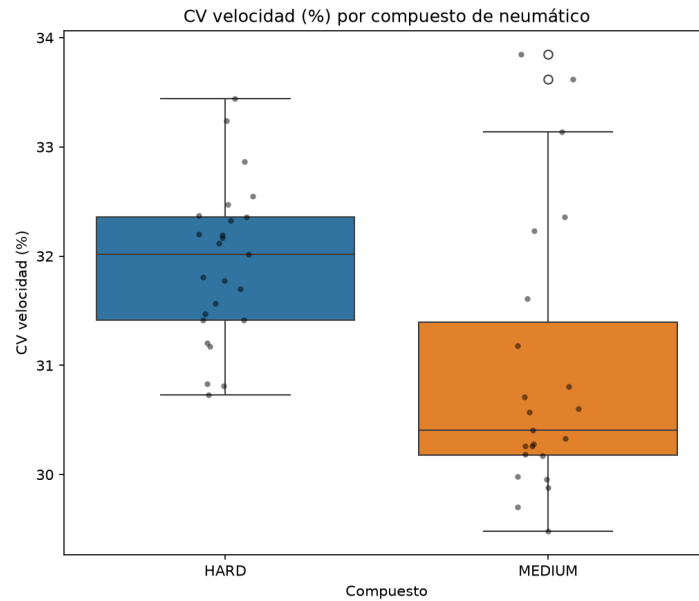


Figura 4.14. CV de velocidad por compuesto de neumático.

4.4.9 Artefactos generados y limitaciones

El pipeline (orquestrado por `Analisis_variabilidad/main.py`, sin argumentos obligatorios más allá del piloto de referencia) persiste una tabla con las 13 métricas calculadas para cada una de las 48 vueltas (`eda/analisis_variabilidad/metricas_variabilidad_por_vuelta.csv`) y 15 figuras en `figures/analisis_variabilidad/`: la distribución conjunta de métricas, un diagrama de Poincaré de ejemplo, y –para cada una de las cuatro métricas clave de la Tabla 4.9– su relación con el tiempo de vuelta, con la vida del neumático y su comparación por compuesto.

Como limitaciones cabe señalar que (i) el análisis se apoya en un único piloto y una única sesión, igual que el resto de fases del proyecto salvo la validación multi-circuito (Sección 4.3.5); (ii) varios parámetros de adaptación (el factor de submuestreo a 5 Hz, el retardo de CONGA, el umbral $k = 1$ de MAGE, los percentiles 25/75 del fenotipo) son elecciones razonables pero no las únicas posibles, y no se ha realizado un análisis de sensibilidad sistemático sobre ellas; y (iii) las correlaciones reportadas en la Tabla 4.9 se calculan sobre solo 48 observaciones (vueltas), por lo que deben interpretarse como indicios –coherentes con la física del problema– más que como resultados estadísticamente concluyentes.

4.5 Modelado ARIMA

4.5.1 Diseño experimental

Tras una primera revisión del planteamiento inicial de este bloque, se rediseñó por completo la evaluación del modelo ARIMA: en lugar de una única partición cronológica train/test sobre una sola vuelta, se plantean **dos experimentos de generalización** independientes, cada uno con su propia búsqueda de hiperparámetros sobre un conjunto de *validación* y una evaluación final sobre un conjunto de *test* disjunto, nunca visto durante la selección del modelo:

- **Experimento 1 (trozo del circuito):** un tramo del trazado se usa para elegir los hiperparámetros y un tramo distinto del mismo trazado para evaluar la generalización espacial del modelo.
- **Experimento 2 (vueltas distintas):** se usa el trazado completo, pero un grupo de vueltas para elegir los hiperparámetros y un grupo de vueltas distinto (cronológicamente posterior) para evaluar la generalización temporal del modelo a lo largo de la carrera.

En ambos casos la partición validación/test mantiene una proporción de 80 % de los datos para validación (búsqueda de hiperparámetros) y 20 % para test (evaluación final).

Con el objetivo de establecer un conjunto explícito de criterios de calidad de la predicción, se adoptan cuatro métricas complementarias, siendo n el número de puntos evaluados, y_i el valor real y $\hat{y}_i$ el valor predicho: el error absoluto medio (MAE), en las unidades originales de la variable predicha,

$$\text{MAE} = \frac{1}{n} \sum_i |y_i - \hat{y}_i|, \quad (4.5)$$

el error cuadrático medio (MSE), en las unidades al cuadrado,

$$\text{MSE} = \frac{1}{n} \sum_i (y_i - \hat{y}_i)^2, \quad (4.6)$$

la raíz del error cuadrático medio (RMSE), que penaliza más los errores grandes y se expresa de nuevo en las mismas unidades que la variable predicha (km/h),

$$\text{RMSE} = \sqrt{\text{MSE}} = \sqrt{\frac{1}{n} \sum_i (y_i - \hat{y}_i)^2}, \quad (4.7)$$

y el coeficiente de determinación,

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}, \quad (4.8)$$

que compara el error del modelo frente al de predecir siempre la media $\bar{y}$ del propio conjunto de test, y permite valorar la calidad del ajuste independientemente de la escala de la variable. Las cuatro métricas se calculan tanto para cada orden candidato durante la búsqueda de hiperparámetros como en la evaluación final sobre el conjunto de test.

El módulo `Modelado_arima` se organiza en cuatro ficheros con responsabilidades independientes: `modelo_arima.py` (ajuste del modelo ARIMA/ARIMAX, búsqueda de hiperparámetros y evaluación walk-forward), `experimentos.py` (construcción de las series de validación/test de ambos experimentos a partir de la telemetría de todas las vueltas, y de la variable exógena $s(t)$ descrita en la Sección 4.5.3), `graficas_arima.py` (generación de las figuras) y `main.py`, el script orquestador —único punto de entrada ejecutable del módulo, coherente con el resto del proyecto— que ejecuta ambos experimentos de principio a fin y persiste tablas y figuras en `arima/` y `figures/arima/`, respectivamente.

Para elegir los hiperparámetros dentro del conjunto de validación se evalúa un pequeño conjunto de órdenes $\text{ARIMA}(p, d, q)$ candidatos —variando el número de rezagos autorregresivos, el orden de diferenciación y la componente de media móvil— mediante validación walk-forward² (*rolling-origin evaluation*) de un paso [34] dentro del propio conjunto de validación (un 80 % interno para ajustar el modelo, un 20 % interno para puntuar cada candidato), y se selecciona el orden con menor RMSE. El conjunto de test no interviene en ningún momento de esta búsqueda.

Elección de los órdenes candidatos.

En un modelo $\text{ARIMA}(p, d, q)$, p es el número de rezagos autorregresivos, d el grado de diferenciación necesario para alcanzar estacionariedad³ y q el orden de la componente de media móvil [22]. La vía clásica para fijar estos tres valores es la identificación visual mediante los correlogramas de autocorrelación (ACF) y autocorrelación parcial (PACF) de la serie —un corte abrupto en la PACF

²La validación *walk-forward* simula el uso real del modelo: se ajusta con los datos disponibles hasta un punto, se predice el siguiente valor, se compara con el dato real cuando llega y se repite el proceso desplazando la ventana de datos un paso hacia delante, sin usar nunca información del futuro para predecir el pasado. “Un paso” se refiere aquí al *horizonte* de la predicción —cuánto futuro se predice de una vez: solo el instante siguiente, $t + 1$ —, no al orden p del modelo, que indica cuánto *pasado* (cuántos rezagos) se usa como entrada. Un modelo $\text{ARIMA}(5, 0, 0)$ predicho a un paso usa, por tanto, los 5 valores anteriores de la serie para producir una única predicción, la del siguiente instante.

³Una serie es *estacionaria* si sus propiedades estadísticas (media, varianza, autocorrelación) no cambian con el tiempo: fluctúa alrededor de un nivel estable, sin tendencias sistemáticas. Los modelos ARIMA asumen esta propiedad; cuando la serie original no la cumple, diferenciarla —restar cada valor menos el anterior, tantas veces como indique d — suele eliminar la tendencia y aproximarla a una serie estacionaria.

sugiere el orden p , un corte abrupto en la ACF sugiere el orden q — comparando después los modelos candidatos mediante un criterio de información como el AIC o el BIC⁴ [22]. En este trabajo se optó por una identificación empírica en su lugar: la serie de velocidad submuestreada es telemetría real de alta frecuencia, con una dinámica no lineal fuertemente condicionada por el trazado (frenadas, curvas, rectas), cuya ACF/PACF no exhibe necesariamente los cortes nítidos de los ejemplos de manual (Figura 4.15); y, al perseguir un objetivo predictivo, el error de generalización medido por validación walk-forward sobre datos nunca vistos —la métrica que en última instancia interesa— es más directamente informativo que una penalización analítica por número de parámetros como el AIC/BIC.

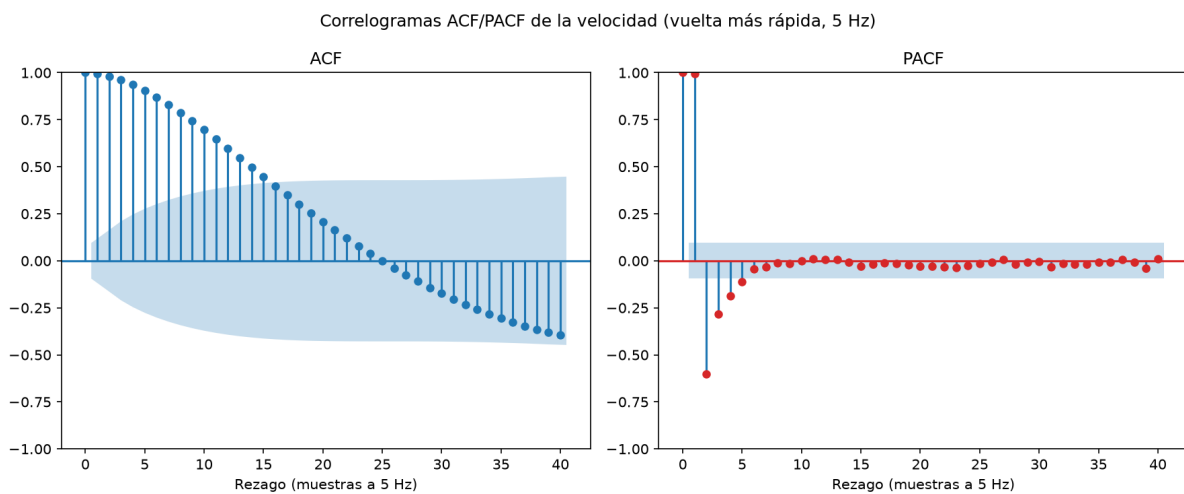


Figura 4.15. Correlogramas ACF y PACF de la velocidad (vuelta más rápida de la sesión de referencia, submuestreada a 5 Hz, 40 rezagos). La ACF decae de forma gradual, sin un corte abrupto; la PACF, tras el primer rezago, muestra varios valores decrecientes pero no despreciables (rezagos 2–5) antes de estabilizarse cerca de cero, en lugar del corte limpio de un ejemplo de manual.

El conjunto de seis candidatos $-(2, 1, 0), (3, 1, 0), (5, 1, 0), (8, 1, 0), (5, 1, 1)$ y $(5, 0, 0)$ — se construyó variando un eje a la vez para mantener acotado el coste computacional del walk-forward (Sección 4.5.6). Primero se barrió el orden autorregresivo $p \in \{2, 3, 5, 8\}$ manteniendo fijos $d = 1$ (una diferenciación de primer orden, la técnica de estacionarización más simple y habitual [22], razonable dado que la velocidad dentro de una vuelta presenta tendencias locales marcadas por frenadas y aceleraciones sostenidas) y $q = 0$, cubriendo así, a la frecuencia de 5 Hz empleada, memorias de

⁴El Criterio de Información de Akaike (AIC) y el Criterio de Información Bayesiano (BIC) comparan modelos candidatos combinando su bondad de ajuste (verosimilitud) con una penalización por el número de parámetros que usan, favoreciendo así modelos que expliquen bien los datos sin ser innecesariamente complejos.

entre 0.4 s y 1.6 s. A partir del mejor p resultante de ese barrido ($p = 5$), se probaron dos variantes puntuales: añadir un término de media móvil $((5, 1, 1))$, y eliminar la diferenciación $((5, 0, 0))$ para comprobar si la componente autorregresiva por sí sola –sin diferenciar– podía igualar o mejorar el ajuste pese a esas tendencias locales. Como se discute en la Sección 4.5.6, es precisamente este último candidato, ARIMAX(5, 0, 0), el que resulta seleccionado de forma consistente en ambos experimentos.

Una vez fijado el orden, el modelo se reajusta sobre *todo* el conjunto de validación y se evalúa mediante validación walk-forward sobre el conjunto de test: para cada instante se predice un paso adelante a partir de los datos anteriores, sin reestimar los parámetros del modelo. A diferencia de un bucle explícito de predicción + reinyección del dato real (computacionalmente inviable para series de varios miles de puntos, al volver a filtrar toda la serie acumulada en cada paso), esta evaluación se implementa mediante el método `extend` de `statsmodels` [35], que continúa el filtro de Kalman⁵ desde el estado final del ajuste y filtra todo el conjunto de test en una única pasada, produciendo en `fittedvalues` la predicción a un paso en cada instante.

4.5.2 Preparación de los datos

Los artefactos `telemetry_piloto.csv` / `telemetry_piloto_interpolada.csv` generados en la fase de extracción y limpieza (Sección 4.1) solo contienen las tres mejores vueltas del piloto de referencia, suficientes para el análisis exploratorio y el análisis del trazado, pero insuficientes para el Experimento 2, que necesita un número considerable de vueltas independientes para poder dividirlos en un grupo de validación y otro de test. Por ello se añadió a `Extraccion_limpieza/main.py` un paso adicional al final del *pipeline* que reutiliza las mismas funciones de extracción y regularización ya empleadas para `telemetry_piloto` (`extraer_telemetria`, `limpiar_telemetria`, `interpolar_telemetria_temporal`) pero sin limitar el número de vueltas por piloto, generando dos artefactos nuevos e independientes en un subdirectorío propio dentro de `csv_data/` (`csv_data/arima/telemetry_todas_vueltas.csv` y su versión regularizada a 100 Hz), que `Modelado_arima` lee como entrada (`--data-dir`, por defecto `csv_data/arima/`) sin alterar los artefactos ya existentes.

⁵`statsmodels` representa internamente el modelo ARIMA como un modelo de espacio de estados: en lugar de manejar directamente los últimos rezagos de la serie, mantiene un *estado* –un resumen compacto de la información del pasado necesaria para predecir el siguiente valor–. El filtro de Kalman es el algoritmo recursivo que actualiza ese estado con cada dato nuevo que llega y produce, de paso, la predicción a un paso, sin necesidad de reprocesar toda la serie acumulada desde el principio.

Para la sesión de referencia (GP de Italia 2023, carrera) se obtuvieron así **48 vueltas limpias** del piloto (de las 49 registradas en `laps_clean.csv`; una vuelta se descarta por no superar el filtro de vueltas rápidas de FastF1), con un total de 416 781 puntos de telemetría regularizada a 100 Hz. Dado que ajustar un modelo ARIMA y realizar la búsqueda de hiperparámetros sobre un volumen de datos de ese orden no es computacionalmente viable en un tiempo razonable, la serie se submuestreó a 5 Hz ($\Delta t = 0.2$ s, tomando 1 de cada 20 puntos dentro de cada vuelta) antes de construir los conjuntos de los dos experimentos, conservando el muestreo regular que exige la metodología de Box-Jenkins [21], solo que con un paso mayor. Tras el submuestreo quedan 20 863 puntos disponibles.

4.5.3 Variable exógena: Índice de Complejidad de Curva $s(t)$

Siguiendo el planteamiento original del bloque de modelado –predecir $g(t + 1)$ a partir no solo de los valores pasados de la propia velocidad, $g(t - h), \dots, g(t)$, sino también de los de la dificultad del trazado en el lugar donde se encuentra el piloto, $s(t - h), \dots, s(t)$ –, se incorporó como **variable exógena** del modelo el **Índice de Complejidad de Curva (ICC)**, ya definido y calculado en la fase de extracción (Sección 4.3) como la curvatura geométrica local sin signo de la reconstrucción del trazado,

$$s(t) = \text{ICC}(s) = \kappa(s) = \frac{|x'(s)y''(s) - y'(s)x''(s)|}{(x'(s)^2 + y'(s)^2)^{3/2}}, \quad (4.9)$$

persistida como columna ICC de `curvatura_interpolada.csv`, indexada por la distancia de arco recorrida s . No se define ninguna fórmula nueva de dificultad para este módulo: se reutiliza directamente el índice ya establecido en el resto del proyecto, evitando así una segunda noción de “dificultad” (además de la variante recortada y normalizada usada solo con fines de visualización en la Figura 4.6) que sería redundante y podría inducir a confusión. Incorporar $s(t)$ da lugar a un **ARIMAX**: el mismo proceso autorregresivo integrado de media móvil sobre $g(t)$, pero cuyo ajuste incluye además, en cada instante, el valor de este regresor externo no generado por el propio modelo.

La distancia de arco s de `curvatura_interpolada.csv` y la distancia recorrida `Distance` de la telemetría miden la misma magnitud física desde el inicio de vuelta, pero a resoluciones distintas y sin coincidir necesariamente en los mismos puntos exactos. Por ello, $s(t)$ se obtiene por **interpolación lineal** del ICC ya calculado en las distancias muestreadas por la telemetría (`experimentos.py:añadir_icc`), de forma que $g(t)$ y $s(t)$ quedan regularizadas al mismo intervalo de muestreo, tal y como exige su uso conjunto en un modelo ARIMAX. Al ser el trazado físicamente el mismo en todas las vueltas de la sesión, una única interpolación por distancia (independiente del número de vuelta) es válida para las 48 vueltas disponibles.

En la práctica, incorporar $s(t)$ no requirió cambiar de clase de modelo: `statsmodels.tsa.arima.model.ARIMA` —la misma clase ya empleada— admite de forma nativa un parámetro `exog`, al estar implementada internamente sobre el mismo motor de espacio de estados que `SARIMAX`. Si fue necesario, en cambio, un paso adicional de **tipificación** (z -score) del ICC antes de pasarlo como regresor. La velocidad y el ICC difieren en varios órdenes de magnitud ($g(t) \sim 10^2$ km/h frente a $ICC \sim 10^{-4} \text{ m}^{-1}$), y esta disparidad de escalas resultaba numéricamente inestable para el optimizador de máxima verosimilitud de `statsmodels`: al buscar iterativamente los coeficientes óptimos, una diferencia de escala tan grande entre regresores deforma la superficie de error que el optimizador debe recorrer, de modo que un mismo tamaño de paso resulta adecuado para una variable pero desproporcionado para la otra, dificultando la convergencia. Se trata de un problema puramente numérico de condicionamiento del optimizador, no de una limitación del ICC como variable explicativa.

La tipificación —restar la media y dividir por la desviación típica— lleva ambas variables a una escala comparable (centradas en 0, con dispersión aproximadamente unitaria), sin alterar la información que aporta el ICC sobre la dificultad de cada punto del trazado: solo cambia la escala en la que se presenta al optimizador. Para evitar fuga de información desde el conjunto de test, la media y la desviación típica empleadas en la tipificación se calculan **únicamente sobre el conjunto de entrenamiento de cada partición walk-forward**, nunca sobre el de test; esos mismos dos estadísticos, ya fijados, se aplican después para tipificar tanto el conjunto de entrenamiento como el de test. De este modo el conjunto de test nunca interviene en el cálculo de los estadísticos de normalización, con la misma disciplina de validación/test que rige el resto del experimento. El resto del cambio se limitó a propagar la serie de ICC tipificada, dividida con exactamente el mismo criterio y en los mismos puntos que la serie de velocidad, tanto en el ajuste inicial (`ARIMA(train, exog=exog_train, order=order)`) como en la extensión walk-forward sobre el conjunto de test (`resultado.extend(test, exog=exog_test)`), preservando íntegramente la disciplina de validación/test de la Sección 4.5.1: $s(t)$ del conjunto de test nunca interviene en la búsqueda de hiperparámetros, exactamente igual que $g(t)$ del conjunto de test.

4.5.4 Experimento 1: generalización espacial (trozo del circuito)

Para cada una de las 48 vueltas se toma el primer 80 % de su distancia recorrida como tramo de validación y el 20 % restante (el final de la vuelta) como tramo de test, y se agregan (concatenan) los tramos correspondientes de todas las vueltas disponibles. El resultado son 17 105 puntos de validación

y 3 758 puntos de test.

La Figura 4.16 muestra el MAE y el RMSE, y la Figura 4.17 el R^2 , de cada orden ARIMAX candidato –ya con $s(t)$ como variable exógena (Sección 4.5.3)– evaluado sobre el conjunto de validación. Las diferencias entre candidatos son pequeñas (RMSE entre 2.66 y 2.73 km/h; R^2 en torno a 0.9988 en todos los casos), lo que indica que el resultado es poco sensible a la elección exacta de (p, d, q) en este experimento; el orden seleccionado es ARIMAX(5, 0, 0).

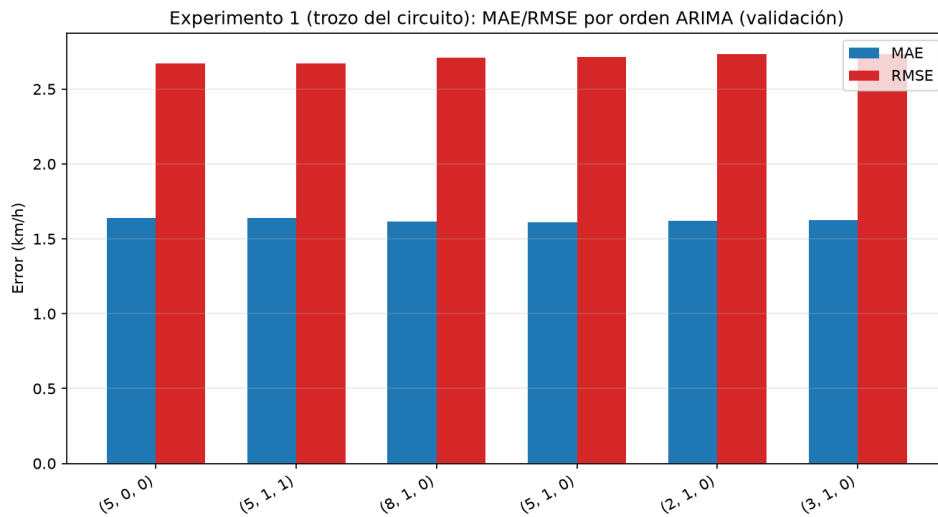


Figura 4.16. Experimento 1: MAE y RMSE de cada orden ARIMAX candidato sobre el conjunto de validación.

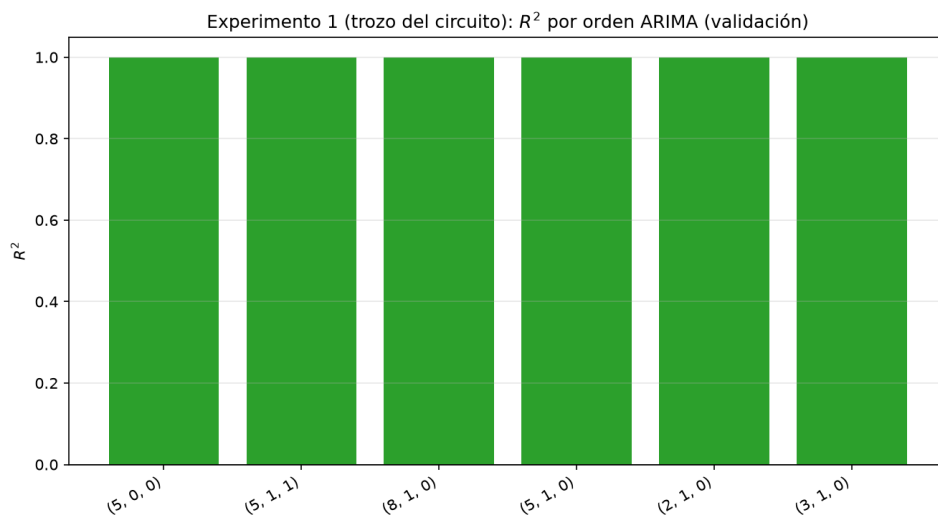


Figura 4.17. Experimento 1: coeficiente de determinación R^2 de cada orden ARIMAX candidato sobre el conjunto de validación.

Con este orden fijado, el modelo se reajusta sobre todo el tramo de validación y se evalúa sobre el

tramo de test (un tramo del circuito nunca visto durante la búsqueda), obteniendo $MAE = 1.452$ km/h, $MSE = 6.644$, $RMSE = 2.578$ km/h y $R^2 = 0.9973$. El error absoluto (MAE , $RMSE$) en test es, de hecho, ligeramente inferior al obtenido en validación, lo que indica que el modelo generaliza bien espacialmente: la dinámica cinemática capturada en un tramo del circuito (frenadas, curvas, rectas) es representativa del resto del trazado, como cabía esperar en un circuito físicamente estable a lo largo de la sesión. El R^2 , sin embargo, es ligeramente inferior en test (0.9973) que en validación (≈ 0.9988): al ser una métrica relativa a la varianza de cada conjunto, un tramo de test con menor variabilidad intrínseca de velocidad exige un ajuste relativamente mejor para alcanzar el mismo R^2 , aunque el error absoluto sea menor en términos de km/h. Este contraste ilustra precisamente el valor de emplear varias métricas complementarias en lugar de una sola. La Figura 4.18 compara visualmente esta predicción a un paso con la velocidad real a lo largo del tramo de test.

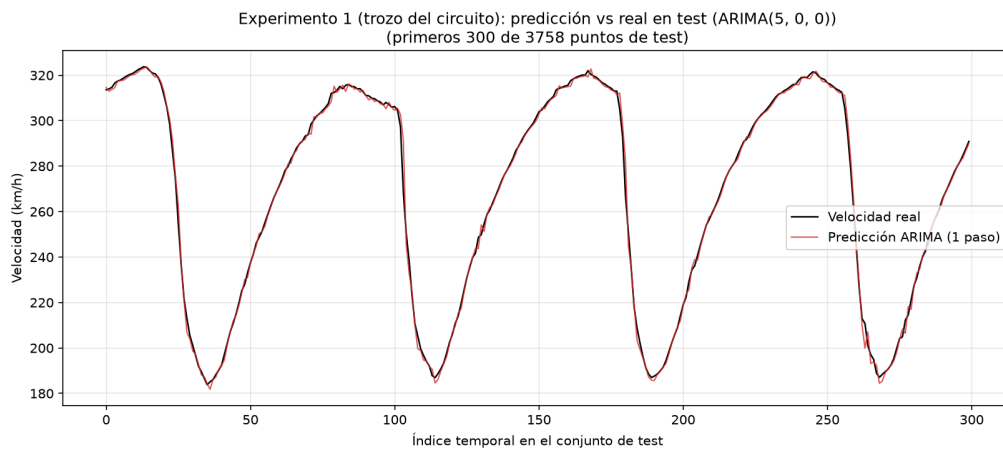


Figura 4.18. Experimento 1: predicción ARIMAX a un paso frente a la velocidad real en el tramo de test.

4.5.5 Experimento 2: generalización temporal (vueltas distintas)

Las 48 vueltas disponibles, ordenadas cronológicamente por número de vuelta, se dividen en un grupo de validación con las 38 primeras (vueltas 2 a 41) y un grupo de test con las 10 últimas (vueltas 42 a 51), cada una con su recorrido completo del circuito. El resultado son 16 544 puntos de validación y 4 319 puntos de test. La Figura 4.19 muestra el MAE y el $RMSE$, y la Figura 4.20 el R^2 , de cada orden ARIMAX candidato evaluado sobre el conjunto de validación.

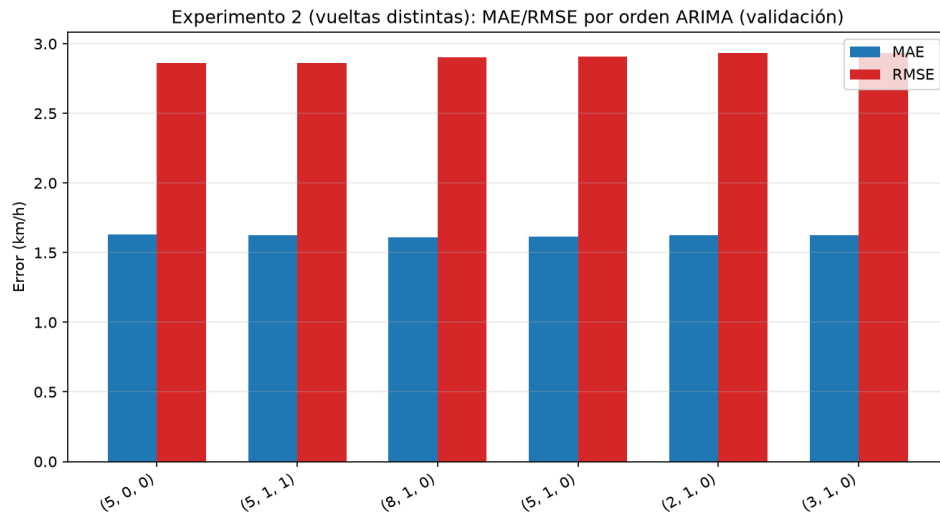


Figura 4.19. Experimento 2: MAE y RMSE de cada orden ARIMAX candidato sobre el conjunto de validación.

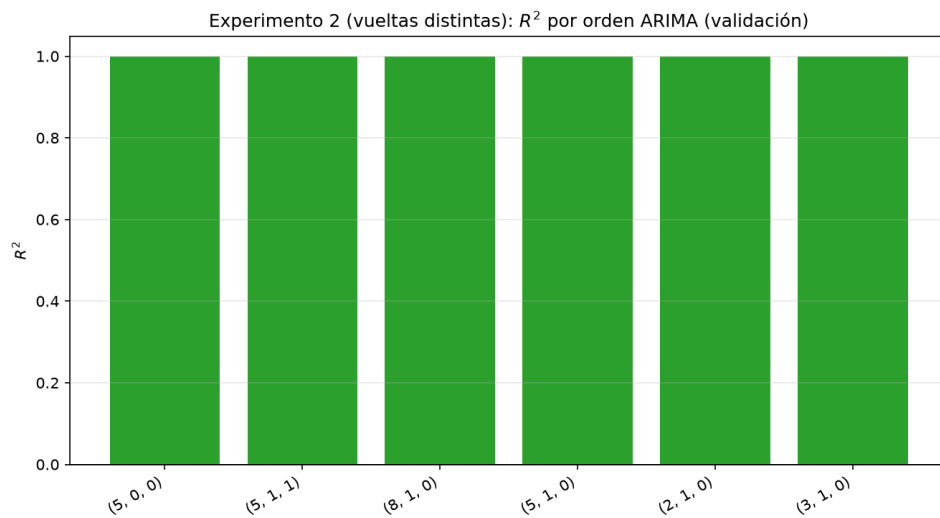


Figura 4.20. Experimento 2: coeficiente de determinación R^2 de cada orden ARIMAX candidato sobre el conjunto de validación.

Como en el experimento anterior, el orden seleccionado en validación es ARIMAX(5, 0, 0) (RMSE = 2.859 km/h, $R^2 = 0.9985$). Evaluado sobre el grupo de test –vueltas finales de la carrera, nunca vistas durante la búsqueda– se obtiene MAE = 1.626 km/h, MSE = 9.437, RMSE = 3.072 km/h y $R^2 = 0.9983$. La Figura 4.21 compara visualmente esta predicción a un paso con la velocidad real a lo largo del grupo de vueltas de test.

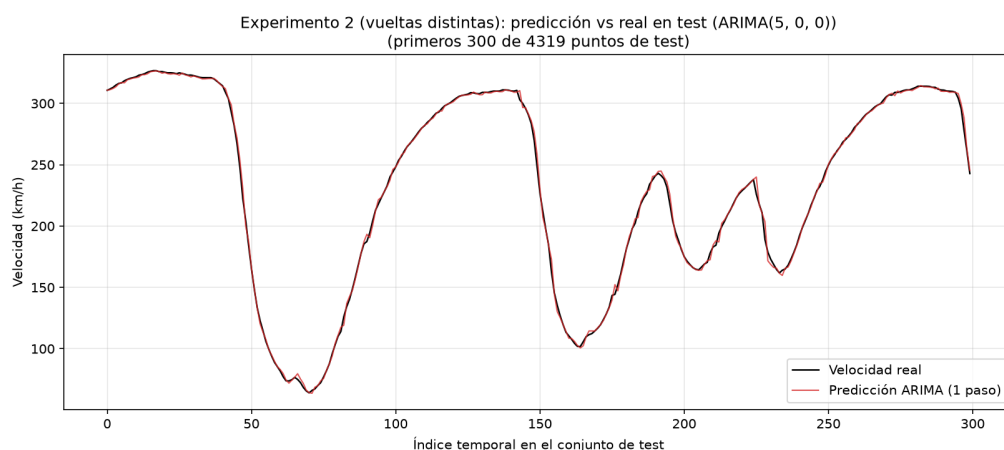


Figura 4.21. Experimento 2: predicción ARIMAX a un paso frente a la velocidad real en el grupo de vueltas de test.

A diferencia del Experimento 1, aquí el error en test es *superior* al obtenido en validación: el modelo generaliza algo peor a vueltas cronológicamente posteriores a las usadas para elegir los hiperparámetros. Es un resultado razonable desde el punto de vista físico: mientras que el trazado no cambia de una vuelta a otra, el comportamiento del monoplaza sí evoluciona a lo largo de la carrera (degradación de los neumáticos, reducción de la masa por consumo de combustible, tráfico), por lo que un modelo ajustado únicamente con las vueltas iniciales pierde algo de precisión al aplicarse a las vueltas finales.

4.5.6 Comparación y conclusiones

La Tabla 4.10 resume los resultados finales de ambos experimentos sobre su respectivo conjunto de test.

Tabla 4.10. Resumen comparativo de los dos experimentos de generalización. Ambos modelos son $ARIMAX(p, d, q)$ con la dificultad del trazado $s(t)$ (Sección 4.5.3) como variable exógena.

Experimento	Orden elegido	MAE test (km/h)	RMSE test (km/h)	R^2 test	Puntos test
1. Trozo del circuito	ARIMAX(5,0,0)	1.452	2.578	0.9973	3 758
2. Vueltas distintas	ARIMAX(5,0,0)	1.626	3.072	0.9983	4 319

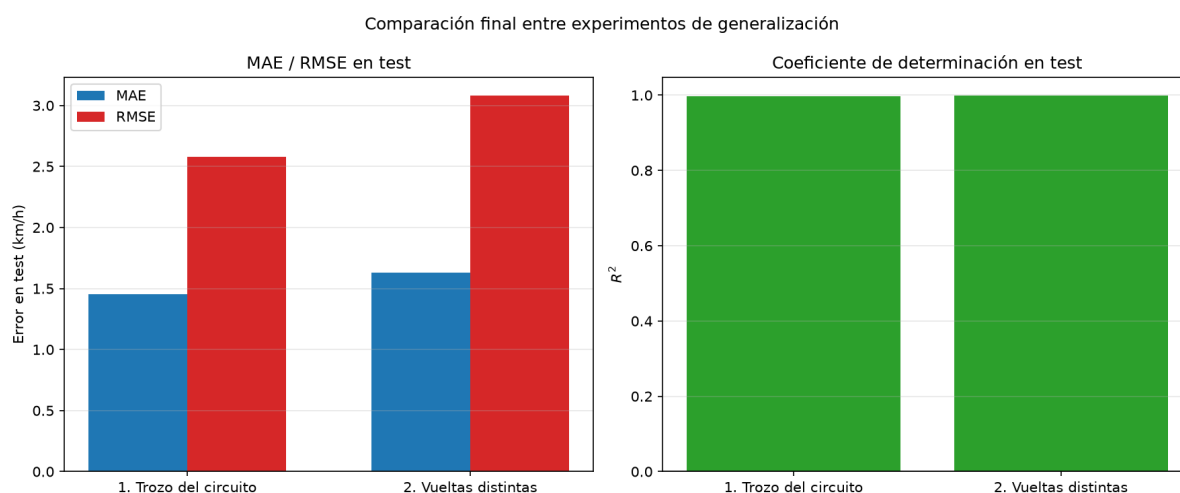


Figura 4.22. Comparación del error final (MAE, RMSE) y del R^2 en test entre ambos experimentos de generalización.

La Figura 4.22 representa gráficamente esta misma comparación. Ambos experimentos coinciden en seleccionar $\text{ARIMAX}(5, 0, 0)$ como mejor orden —sin componente de diferenciación ($d = 0$) ni de media móvil—, lo que sugiere que, a la frecuencia de muestreo empleada (5 Hz), la velocidad submuestreada se comporta como un proceso suficientemente estacionario en el corto plazo como para no requerir diferenciación explícita. En error absoluto (MAE, RMSE), el Experimento 1 generaliza mejor que el 2 (1.452 vs. 1.626 km/h de MAE): el modelo tolera mejor un tramo de circuito no visto que vueltas posteriores no vistas, lo que apunta a que la variable dominante que degrada la predicción a un paso no es la posición en el trazado, sino la evolución temporal del ritmo de carrera. El R^2 , sin embargo, invierte este orden (0.9973 en el Experimento 1 frente a 0.9983 en el 2): como se explica en la Sección 4.5.4, el tramo de test del Experimento 1 tiene menor variabilidad intrínseca de velocidad, lo que penaliza su R^2 pese a presentar el menor error absoluto de los dos experimentos. En conjunto, ambas métricas cuentan una historia consistente y complementaria: el Experimento 1 comete errores más pequeños en valor absoluto, pero el Experimento 2 explica una proporción ligeramente mayor de la varianza de su propio conjunto de test.

El ICC $s(t)$ aporta al modelo información explícita sobre la posición en el trazado, pero su contribución al error final es **pequeña**: los rezagos autorregresivos de la propia velocidad, $g(t - h), \dots, g(t)$, ya capturan implícitamente buena parte de esa información posicional —frenar y acelerar en una curva deja una huella característica en la propia serie de velocidad—, por lo que la señal explícita que aporta el ICC resulta en gran medida redundante con la que el modelo ya puede inferir de $g(t)$ a través de sus propios rezagos. No es un fallo de la variable, sino una consecuencia esperable de introducir un

regresor posicional en un modelo ya autorregresivo sobre una serie con una dinámica tan marcada por el trazado.

Como limitaciones de este diseño experimental, cabe señalar que (i) los conjuntos de validación y test se construyen concatenando tramos o vueltas de telemetría discontinuos entre sí, lo que introduce puntos de unión artificiales (con un salto de velocidad no realista) en un número reducido de instantes, sin impacto apreciable en las métricas agregadas dado el volumen de puntos; y (ii) el submuestreo a 5 Hz, necesario para que la búsqueda de hiperparámetros fuese computacionalmente viable, limita la capacidad del modelo para capturar transitorios muy rápidos (frenadas de menos de 0.2 s).

5

Conclusiones

5.1 Cumplimiento de los objetivos

El anteproyecto de este TFG planteaba cinco objetivos específicos. A la luz del trabajo desarrollado y descrito en el Capítulo 4, pueden darse todos ellos por cumplidos:

- **Acceder, limpiar y organizar los datos de vueltas de Fórmula 1 mediante FastF1.** El módulo `Extraccion_limpieza` implementa un *pipeline* completo y reproducible que extrae telemetría, tiempos de vuelta, geometría del circuito y condiciones meteorológicas de la sesión de referencia (GP de Italia 2023), con trazabilidad completa y validado también sobre otras cuatro sesiones (Sección 4.3.5).
- **Analizar el rendimiento de pilotos y escuderías, extrayendo estadísticas y visualizaciones.** El módulo `Analisis_exploratorio` genera 15 figuras y 4 tablas resumen sobre ritmo de carrera, telemetría y condiciones meteorológicas (Sección 4.2).
- **Estudiar la variabilidad de la velocidad y su relación con la dificultad del trazado.** Se caracterizó la dificultad geométrica de cada curva (Sección 4.3) y se validó cuantitativamente su relación con el *nivel* de velocidad real del piloto, obteniendo una correlación de Pearson $r = 0.804$ entre el radio de curvatura y la velocidad mínima real de paso (Sección 4.3.4). Por otro lado, se caracterizó en profundidad la variabilidad de la velocidad *en el tiempo dentro de una vuelta*, con un conjunto de 13 métricas adaptadas de la literatura de variabilidad glucémica (convencionales, de secuencialidad, de fenotipo y de análisis de fluctuación sin tendencia) aplicadas al perfil de velocidad de cada una de las 48 vueltas disponibles; estas métricas se relacionan con el ritmo de vuelta y la vida del neumático más que con la dificultad geométrica, que

—al ser la misma para las 48 vueltas de un único circuito— no varía entre vueltas y no puede correlacionarse a ese nivel de agregación (Sección 4.4).

- **Implementar un modelo predictivo ARIMA para estimar la velocidad a partir de valores pasados.** Se implementó, incorporando el Índice de Complejidad de Curva $s(t)$ como variable exógena —de ARIMA a ARIMAX (Sección 4.5.3)— y se sometió a una validación más exigente que la inicialmente prevista: dos experimentos de generalización independientes (por tramo del circuito y por vuelta), con búsqueda de hiperparámetros restringida al conjunto de validación y evaluación final sobre un conjunto de test nunca visto (Sección 4.5).
- **Generar un informe completo con las conclusiones obtenidas.** Es el objeto del presente capítulo.

5.2 Principales hallazgos

Extracción y limpieza. El *pipeline* sobre FastF1 resultó robusto, pero su validación reveló un error real de escala (un factor $10\times$ en las coordenadas X/Y del canal de posición, en decímetros y no en metros) que distorsionaba toda la geometría reconstruida del circuito; su corrección (Sección 4.3) ilustra la importancia de contrastar cada artefacto generado con una estimación independiente antes de darlo por válido. La comparativa de frecuencias de interpolación confirmó, con datos, que los 100 Hz empleados por defecto son necesarios y no excesivos frente a alternativas más gruesas (Sección 4.1.6.3).

Análisis exploratorio. Las estadísticas descriptivas y las visualizaciones generadas permiten caracterizar el ritmo de carrera y el comportamiento cinemático del monoplaza: en la sesión de referencia, el piloto con mejor tiempo de vuelta mediano (Max Verstappen, 86.05 s) queda seguido muy de cerca por otros cuatro pilotos, todos con medianas por debajo de 86.6 s, evidenciando un grupo de cabeza muy igualado en ritmo puro (Sección 4.2).

Dificultad del trazado. La segmentación geométrica identificó 11 curvas en el trazado de referencia, coincidentes en número y posición relativa con las curvas reales de Monza, y su relación con la velocidad real del piloto quedó validada cuantitativamente ($r = 0.804$). La validación multi-circuito de la reconstrucción arrojó un resultado más matizado de lo inicialmente hipotetizado: el método generaliza correctamente a tres de los cuatro circuitos adicionales probados (Spa-Francorchamps, Silverstone y, contrariamente a lo esperado, también Singapur), y solo se degrada claramente en Mónaco, por sobre-segmentación de curvas extremadamente cerradas con un umbral fijo calibrado para

Monza (Sección 4.3.5).

Análisis de variabilidad de la velocidad. La adaptación de métricas de variabilidad glucémica al perfil de velocidad intra-vuelta (Sección 4.4) reveló dos patrones distintos y complementarios sobre las 48 vueltas de la sesión de referencia: por un lado, el CV de velocidad correlaciona con el tiempo de vuelta ($r = 0.670$), es decir, las vueltas más lentas son también las más irregulares; por otro, el exponente DFAint y la componente de corto plazo del diagrama de Poincaré (SD1) correlacionan, en cambio, con la vida del neumático ($r = -0.521$ y $r = 0.498$) casi independientemente del tiempo de vuelta, sugiriendo que capturan un efecto de la degradación del neumático sobre el estilo de conducción distinto del ya conocido efecto sobre el ritmo.

Modelado ARIMA. Los dos experimentos de generalización, con el Índice de Complejidad de Curva $s(t) = ICC(s)$ incorporado como variable exógena tipificada (ARIMAX), seleccionan de forma consistente el mismo orden, ARIMAX(5, 0, 0), y alcanzan un R^2 superior a 0.997 en ambos casos. La contribución de $s(t)$ al error final es pequeña –indicio de que la información posicional que aporta es en buena parte redundante con la que los propios rezagos de $g(t)$ ya capturaban–. El experimento basado en un tramo de circuito no visto generaliza mejor en error absoluto (MAE, RMSE) que el basado en vueltas posteriores no vistas, lo que apunta a que la evolución temporal del ritmo de carrera –más que la posición en el trazado– es la variable que más dificulta la predicción a un paso. El R^2 , sin embargo, matiza esta conclusión al depender de la varianza de cada conjunto de test, evidenciando el valor de emplear varias métricas complementarias en lugar de una sola (Sección 4.5.6).

5.3 Limitaciones

El trabajo presenta limitaciones que conviene explicitar: (i) el análisis detallado de rendimiento y la relación velocidad-dificultad se apoyan en un único piloto y en tres vueltas de referencia –lo que hace que la correlación entre radio de curvatura y velocidad mínima real ($r = 0.804$, Sección 4.3.4) se calcule sobre solo $n = 11$ curvas– aunque los experimentos de generalización de ARIMA sí se extendieron a las 48 vueltas limpias disponibles; (ii) el umbral de segmentación de curvas (300 m) y el suavizado del *spline* de reconstrucción (80) están calibrados para las proporciones de Monza y no se adaptan automáticamente a la escala de otros circuitos, lo que explica el resultado de Mónaco; (iii) los experimentos ARIMA se evaluaron a 5 Hz por el coste computacional de la búsqueda de hiperparámetros, lo que limita la capacidad del modelo para capturar transitorios muy rápidos; y (iv) todo el trabajo

se circunscribe a una única temporada y a un conjunto reducido de circuitos; y (v) las métricas de variabilidad de la Sección 4.4 dependen de varias elecciones de adaptación (factor de submuestreo, retardo de CONGA, umbral de MAGE, percentiles del fenotipo) no sometidas a un análisis de sensibilidad sistemático, y sus correlaciones con el tiempo de vuelta y la vida del neumático se calculan sobre solo 48 vueltas.

5.4 Líneas futuras

A partir de las limitaciones anteriores y de los propios resultados obtenidos, se identifican varias líneas de continuación naturales para este trabajo:

Se listan a continuación en el mismo orden que las limitaciones (i)–(v) de la Sección 5.3 a las que responde cada una, más dos líneas adicionales motivadas directamente por los hallazgos del Capítulo 4:

- **Ampliar la base de pilotos y vueltas de referencia** (responde a la limitación (i)). El análisis detallado de rendimiento se apoya en un único piloto y tres vueltas de referencia; extenderlo a varios pilotos de la misma sesión permitiría comprobar si la relación entre radio de curvatura y velocidad mínima real ($r = 0.804$, Sección 4.3.4) se mantiene con más pilotos y, por tanto, más curvas observadas ($n = 11$ en la configuración actual).
- **Umbral de segmentación adaptativo** (responde a la limitación (ii)). El fallo de la reconstrucción en Mónaco (Sección 4.3.5) apunta directamente a sustituir el umbral fijo de 300 m y el suavizado fijo del *spline* por valores proporcionales a alguna medida de escala del circuito (por ejemplo, su longitud total o el radio medio de sus curvas), de forma que el mismo método se adapte sin recalibración manual a circuitos tan distintos como Monza y Mónaco.
- **Búsqueda de hiperparámetros más eficiente para evaluar a mayor frecuencia** (responde a la limitación (iii)). El submuestreo a 5 Hz se adoptó para que la búsqueda de (p, d, q) sobre los seis candidatos y los dos experimentos fuese computacionalmente viable; técnicas de selección de orden más dirigidas (por ejemplo, un *auto-ARIMA* basado en criterios de información en lugar de una rejilla exhaustiva) permitirían evaluar a frecuencias más altas y así capturar transitorios de frenada por debajo de los 0.2 s que actualmente se pierden.
- **Extender la cobertura a más temporadas y circuitos** (responde a la limitación (iv)). Todo el

trabajo se circunscribe a la temporada 2023 y, en la validación multi-circuito, a cinco trazados; repetir el pipeline completo –no solo la validación geométrica– sobre más temporadas y circuitos permitiría comprobar si conclusiones como el orden ARIMAX(5, 0, 0) seleccionado en ambos experimentos son propias del GP de Italia o generalizan a otros trazados.

- **Análisis de sensibilidad de las métricas de variabilidad** (responde a la limitación (v)). Las elecciones de adaptación de las métricas de variabilidad (Sección 4.4) –factor de submuestreo, retardo de CONGA, umbral de MAGE, percentiles del fenotipo– podrían someterse a un barrido sistemático para comprobar la robustez de las correlaciones reportadas frente a esas decisiones.
- **Otras variables exógenas para ARIMAX.** Dado que la contribución del ICC $s(t)$ al error final es pequeña (Sección 4.5.6) por ser en gran parte redundante con los propios rezagos de la velocidad, cabría explorar variables exógenas con información que el modelo no pueda inferir de su propio pasado: carga de combustible restante, degradación acumulada del neumático o proximidad a otro monoplaza (relevante para el uso del DRS).
- **Modelos alternativos y predicción a varios pasos.** Frente al enfoque interpretable de ARIMA/ARIMAX, sería natural comparar su capacidad predictiva con modelos no lineales (LSTM, Prophet, *transformers* temporales) sobre el mismo diseño experimental de dos experimentos de generalización, y extender la evaluación –limitada en este trabajo a la predicción a un paso ($t + 1$)– a horizontes de varios pasos, más exigente y más útil de cara a aplicaciones de estrategia en tiempo real.

Bibliografía

- [1] Python Software Foundation. "Python logos and branding," Accessed: Feb. 20, 2026. [Online]. Available: <https://www.python.org/community/logos/>
- [2] FastF1 Developers. "Fastf1 core module documentation," Accessed: Feb. 20, 2026. [Online]. Available: <https://docs.fastf1.dev/core.html>
- [3] Wikimedia Commons. "File: Pandas logo," Accessed: Feb. 20, 2026. [Online]. Available: https://commons.wikimedia.org/wiki/File:Pandas_logo.svg
- [4] Wikimedia Commons. "File: Numpy logo 2020," Accessed: Feb. 20, 2026. [Online]. Available: https://commons.wikimedia.org/wiki/File:NumPy_logo_2020.svg
- [5] TechIcons. "Matplotlib icon," Accessed: Feb. 20, 2026. [Online]. Available: <https://techicons.dev/icons/matplotlib>
- [6] Wikimedia Commons. "File: Logo seaborn," Accessed: Feb. 20, 2026. [Online]. Available: <https://commons.wikimedia.org/wiki/File:Logo-seaborn.png>
- [7] Statsmodels Developers. "About statsmodels," Accessed: Feb. 20, 2026. [Online]. Available: <https://www.statsmodels.org/stable/about.html>
- [8] Wikimedia Commons. "File: Pycharm icon," Accessed: Feb. 20, 2026. [Online]. Available: https://commons.wikimedia.org/wiki/File:PyCharm_Icon.svg
- [9] Wikimedia Commons. "File: Latex logo," Accessed: Feb. 20, 2026. [Online]. Available: https://commons.wikimedia.org/wiki/File:LaTeX_logo.svg
- [10] Overleaf. "Overleaf brand logos," Accessed: Feb. 20, 2026. [Online]. Available: <https://www.overleaf.com/for/partners/logos>
- [11] Motorsport.com. "Racewatch, el cerebro digital de la fórmula 1," Accessed: Feb. 20, 2026. [Online]. Available: <https://es.motorsport.com/f1/news/racewatch-cerebro-digital-formula1-guia-fia/10796182/>
- [12] Catapult. "Racewatch – soluciones para el análisis de carreras," Accessed: Feb. 20, 2026. [Online]. Available: <https://www.catapult.com/es/soluciones/reloj-de-carreras>

- [13] Amazon Web Services. "Formula 1 and aws: F1 insights." Asociación F1–AWS activa desde 2018, Accessed: Feb. 20, 2026. [Online]. Available: <https://aws.amazon.com/es/sports/f1/>
- [14] FastF1 Developers. "Fastf1: Formula 1 data analysis in python." Primera versión pública en PyPI (2.1) publicada en marzo de 2021, Accessed: Feb. 20, 2026. [Online]. Available: <https://docs.fastf1.dev>
- [15] Ergast. "Ergast developer api." Servicio discontinuado a finales de 2024; sustituido por el proyecto comunitario Jolpica-F1, Accessed: Feb. 20, 2026. [Online]. Available: <https://ergast.com/mrd/>
- [16] OpenF1. "Openf1 api – open formula 1 data." Ofrece datos desde la temporada 2023, Accessed: Feb. 20, 2026. [Online]. Available: <https://openf1.org>
- [17] F1-Dash. "F1-dash: Interactive formula 1 data dashboard," Accessed: Feb. 20, 2026. [Online]. Available: <https://f1-dash.com>
- [18] F1-Tempo. "F1-tempo: Formula 1 data visualization platform," Accessed: Feb. 20, 2026. [Online]. Available: <https://f1-tempo.com>
- [19] A. R. Patro, R. Sahana, S. Gautam, and R. Sapna, "A predictive analysis of lap time in formula 1 using machine learning," in *2025 IEEE 6th India Council International Subsections Conference (INDISCON)*, Rourkela, India, 2025, pp. 1–7. DOI: [10.1109/INDISCON66021.2025.11252332](https://doi.org/10.1109/INDISCON66021.2025.11252332) [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/11252332>
- [20] A. Jafri, *Predicting formula 1 race outcomes: A machine learning approach*, Capstone Project, Computer Science, New York University Abu Dhabi (NYUAD), Advisors: Djellel Difallah y Talal Rahwan. Presentado en el Capstone Seminar de primavera, 2024. Accessed: Aug. 26, 2026. [Online]. Available: https://aliabdullahjafri.com/static/media/Ali_Jafri_CapstoneProject1_Fall2024.c7244022875d46bec5d9.pdf
- [21] G. E. P. Box and G. M. Jenkins, *Time Series Analysis: Forecasting and Control*. San Francisco: Holden-Day, 1970.
- [22] J. Noble. "¿qué son los modelos arima?" IBM, Accessed: Aug. 12, 2026. [Online]. Available: <https://www.ibm.com/es-es/think/topics/arima-model>

- [23] D. Doyle, G. Cunningham, G. White, and J. Early, "Lap time simulation tool for the development of an electric formula student car," SAE International, SAE Technical Paper 2019-01-1070, 2019. DOI: [10.4271/2019-01-1070](https://doi.org/10.4271/2019-01-1070) [Online]. Available: https://pureadmin.qub.ac.uk/ws/portalfiles/portal/164458711/Lap_Time_Simulation_Tool_for_the_Development_of_an_Electric_Formula_Student_Car.pdf
- [24] A. Colás, L. Vigil, B. Vargas, D. Cuesta-Frau, and M. Varela, "Detrended fluctuation analysis in the prediction of type 2 diabetes mellitus in patients at risk: Model optimization and comparison with other metrics," *PLOS ONE*, vol. 14, no. 12, e0225817, 2019. DOI: [10.1371/journal.pone.0225817](https://doi.org/10.1371/journal.pone.0225817)
- [25] M. Rivero. "Cgm-variability-analysis: Glucose variability metrics analysis pipeline," Accessed: Aug. 10, 2026. [Online]. Available: <https://github.com/martinrivero15/cgm-variability-analysis>
- [26] P. Virtanen, R. Gommers, T. E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau, et al., "SciPy 1.0: Fundamental algorithms for scientific computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020. DOI: [10.1038/s41592-019-0686-2](https://doi.org/10.1038/s41592-019-0686-2)
- [27] W. McKinney, "Data structures for statistical computing in Python," in *Proceedings of the 9th Python in Science Conference*, S. van der Walt and J. Millman, Eds., 2010, pp. 56–61. DOI: [10.25080/Majora-92bf1922-00a](https://doi.org/10.25080/Majora-92bf1922-00a)
- [28] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, et al., "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, 2020. DOI: [10.1038/s41586-020-2649-2](https://doi.org/10.1038/s41586-020-2649-2)
- [29] J. D. Hunter, "Matplotlib: A 2d graphics environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007. DOI: [10.1109/MCSE.2007.55](https://doi.org/10.1109/MCSE.2007.55)
- [30] M. L. Waskom, "Seaborn: Statistical data visualization," *Journal of Open Source Software*, vol. 6, no. 60, p. 3021, 2021. DOI: [10.21105/joss.03021](https://doi.org/10.21105/joss.03021)
- [31] F. J. Service, G. D. Molnar, J. W. Rosevear, E. Ackerman, L. C. Gatewood, and W. F. Taylor, "Mean amplitude of glycemic excursions, a measure of diabetic instability," *Diabetes*, vol. 19, no. 9, pp. 644–655, 1970. DOI: [10.2337/diab.19.9.644](https://doi.org/10.2337/diab.19.9.644)
- [32] M. Brennan, M. Palaniswami, and P. Kamen, "Do existing measures of Poincaré plot geometry reflect nonlinear features of heart rate variability?" *IEEE Transactions on Biomedical Engineering*, vol. 48, no. 11, pp. 1342–1347, 2001. DOI: [10.1109/10.959330](https://doi.org/10.1109/10.959330)

- [33] C.-K. Peng, S. V. Buldyrev, S. Havlin, M. Simons, H. E. Stanley, and A. L. Goldberger, "Mosaic organization of DNA nucleotides," *Physical Review E*, vol. 49, no. 2, pp. 1685–1689, 1994. DOI: [10.1103/PhysRevE.49.1685](https://doi.org/10.1103/PhysRevE.49.1685)
- [34] C. Bergmeir and J. M. Benítez, "On the use of cross-validation for time series predictor evaluation," *Information Sciences*, vol. 191, pp. 192–213, 2012. DOI: [10.1016/j.ins.2011.12.028](https://doi.org/10.1016/j.ins.2011.12.028)
- [35] S. Seabold and J. Perktold, "Statsmodels: Econometric and statistical modeling with Python," in *Proceedings of the 9th Python in Science Conference*, 2010, pp. 92–96. DOI: [10.25080/Majora-92bf1922-011](https://doi.org/10.25080/Majora-92bf1922-011)



UNIVERSIDAD
DE MÁLAGA

| **uma.es**

ETS. de Ingeniería Informática

Bulevar Louis Pasteur, 35

Campus de Teatinos

29071 Málaga